```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py                启动并自动打开浏览器
    python run.py --no-browser   只启动服务，不打开浏览器
    python run.py --dev          开发模式（开启代码热重载与调试器）
    python run.py --port 5001    指定端口
默认以 127.0.0.1:5000 启动，就绪后自动打开浏览器。
"""
import os
import sys
import threading
import time
import webbrowser
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")   # 控制台中文正常显示
from app import create_app
from app.models import db
PORT = 5000
for i, a in enumerate(sys.argv):
    if a == "--port" and i + 1 < len(sys.argv):
        PORT = int(sys.argv[i + 1])
URL = "http://127.0.0.1:%d" % PORT
NO_BROWSER = "--no-browser" in sys.argv
DEV = "--dev" in sys.argv
app = create_app()
def open_browser_later(delay=1.2):
    """等服务工作起来再打开浏览器，避免打开时页面还没就绪。
    放在后台线程里，不用 sleep 阻塞服务启动；线程设为守护线程，
    用户 Ctrl+C 停止服务时不会残留。
    """
    def run():
        time.sleep(delay)
        try:
            webbrowser.open(URL)
        except Exception:
            pass            # 打开失败不影响服务本身
    t = threading.Thread(target=run, daemon=True)
    t.start()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()     # 首次运行时自动建表
        from app.utils.schema import ensure_schema
        added = ensure_schema(db)     # 旧库补齐新增字段
        if added:
            print("已补充字段：%s" % "、".join(added))
    print("=" * 58)
    print("   历史遗留废弃矿山自然恢复适宜性评价系统  Ｖ１．０")
    print("=" * 58)
```

```
    print("   访问地址：%s" % URL)
    print("   演示账号：admin      密码：admin123")
    print("   停止运行：按 Ctrl+C，或直接关闭本窗口")
    print("=" * 58)
    if not NO_BROWSER and not DEV:
        print("   正在打开浏览器……")
    print()
    #  开发模式才启用热重载（重载会二次启动进程，故此时不自动开浏览器）
    if not NO_BROWSER and not DEV:
        open_browser_later()
    app.run(host="127.0.0.1", port=PORT, debug=DEV,
    use_reloader=DEV)
#  ==== app/__init__.py ====
#  -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from flask import Flask
from flask_login import LoginManager
from app.config import Config
from app.models import db, User
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    #  注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report
    app.register_blueprint(auth.bp)
    app.register_blueprint(project.bp)
    app.register_blueprint(unit.bp)
    app.register_blueprint(standard.bp)
    app.register_blueprint(constraint.bp)
    app.register_blueprint(ahp.bp)
    app.register_blueprint(indicator.bp)
    app.register_blueprint(evaluate.bp)
    app.register_blueprint(report.bp)
    #  全局登录保护：除登录页与静态资源外，均需登录
    from flask import request, redirect, url_for
    from flask_login import current_user, login_user
    PUBLIC_ENDPOINTS = {"auth.login", "static"}
    @app.before_request
    def require_login():
        #  截图模式（仅用于生成说明书配图）：自动以管理员身份登录
```

```
    if app.config.get("SCREENSHOT_MODE"):
        if (request.endpoint not in ("auth.login",
"static")
                and not current_user.is_authenticated):
            admin = User.query.filter_by(role="admin").f
irst()
            if admin:
                login_user(admin)
        return None
    if request.endpoint in PUBLIC_ENDPOINTS or
request.endpoint is None:
        return None
    if not current_user.is_authenticated:
        return redirect(url_for("auth.login"))
    return None
# 侧边栏导航：按论文评价流程的顺序编排，分为「评价流程」与「系统管理」两组。
# 不使用模块编号，避免出现与实际流程不一致的代号。
@app.context_processor
def inject_nav():
    path = request.path or "/"
    flow = [
        ("项目与矿山档案", "/project/", path.startswith("/proje
ct")),
        ("政策与标准知识库", "/standard/",
path.startswith("/standard")),
        ("限制性指标判定", "/constraint/",
path.startswith("/constraint")),
        ("微地质单元", "/unit/", path.startswith("/unit")),
        ("指标体系与权重", "/ahp/", path.startswith("/ahp")),
        ("指标数据采集", "/indicator/",
path.startswith("/indicator")),
        ("适宜性定量评价", "/evaluate/",
path.startswith("/evaluate")),
        ("报告与结果输出", "/report/", path.startswith("/report
")),
    ]
    admin = [
        ("用户与操作日志", "/users", path.startswith(("/users",
"/logs"))),
    ]
    groups = [
        {"title": "评价流程", "links": flow},
        {"title": "系统管理", "links": admin},
    ]
    # 层级引导用：项目是归档单位，图斑才是评价对象。
    # 下游各页面按图斑组织数据，因此需要提示「哪些项目下还没有图斑」。
    counts = {"project_count": 0, "patch_count": 0,
              "projects_without_patch": []}
    if current_user.is_authenticated:
        from app.models import Project, MinePatch
```

```
            projects = Project.query.order_by(Project.id).al
    l()
            counts["project_count"] = len(projects)
            counts["patch_count"] = MinePatch.query.count()
            counts["projects_without_patch"] = [
                p for p in projects if p.patches.count() ==
    0]
        counts["nav_groups"] = groups
        counts["home_active"] = path == "/"
        return counts
    # 操作审计：统一在响应阶段记录成功的写操作（操作日志）
    from app.services import audit
    @app.after_request
    def audit_writes(response):
        if request.method in ("POST", "PUT", "PATCH",
    "DELETE"):
            audit.record(request.endpoint,
    response.status_code)
        return response
    # 错误页：给出可读提示，而不是默认的英文错误页
    from flask import render_template
    @app.errorhandler(403)
    def forbidden(e):
        return render_template("error.html", code=403,
                               message="没有权限执行该操作"), 403
    @app.errorhandler(404)
    def not_found(e):
        return render_template("error.html", code=404,
                               message="请求的页面不存在"), 404
    return app
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
from datetime import timedelta
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY",
    "mine-restoration-dev-key")
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
    # 「记住我」：登录时勾选则下发持久 cookie，关闭浏览器后仍可自动登录
    REMEMBER_COOKIE_DURATION = timedelta(days=30)
    REMEMBER_COOKIE_HTTPONLY = True
    REMEMBER_COOKIE_SAMESITE = "Lax"
    # 影像与数据文件上传（遥感影像挂接、批量导入）
```

```
    UPLOAD_FOLDER = os.path.join(BASE_DIR, "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024          # 单次上传上限
    32 MB
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
    ALLOWED_DATA_EXT = {"csv", "xlsx", "xls"}
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from datetime import datetime
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（用户与权限管理）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 角色：管理员 admin / 评价员 evaluator / 查看者 viewer
    role = db.Column(db.String(20), nullable=False,
    default="viewer")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    def to_dict(self):
        return {"id": self.id, "username": self.username,
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志。"""
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
```

```
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, index=True)
class Project(db.Model):
    """评价项目（项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
    created_by = db.Column(db.Integer,
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    patches = db.relationship("MinePatch",
    backref="project", lazy="dynamic",
                              cascade="all, delete-orphan")
class MinePatch(db.Model):
    """矿山图斑。"""
    __tablename__ = "mine_patches"
    id = db.Column(db.Integer, primary_key=True)
    project_id = db.Column(db.Integer,
    db.ForeignKey("projects.id"), nullable=False)
    name = db.Column(db.String(128), nullable=False)
    admin_region = db.Column(db.String(128), nullable=True)
    # 所在行政区
    center_x = db.Column(db.Float, nullable=True)
    # 中心经度 / 横坐标
    center_y = db.Column(db.Float, nullable=True)
    # 中心纬度 / 纵坐标
    area_ha = db.Column(db.Float, nullable=True)
    # 图斑面积（公顷）
    mineral_type = db.Column(db.String(64), nullable=True)
    # 开采矿种
    mining_method = db.Column(db.String(64), nullable=True)
    # 开采方式
    stop_years = db.Column(db.Float, nullable=True)
    # 停产年限
    responsible_entity_missing = db.Column(db.Boolean,
    default=True)  # 责任主体是否缺失
    # 遥感底图 / 无人机正射影像挂接
    image_path = db.Column(db.String(256), nullable=True)
    # 相对 static 的路径
    image_note = db.Column(db.String(256), nullable=True)
    # 影像说明（如来源、时相）
    image_uploaded_at = db.Column(db.DateTime,
    nullable=True)
    created_at = db.Column(db.DateTime,
```

```
    default=datetime.utcnow)                                          301
    units = db.relationship("MicroUnit", backref="patch",             302
    lazy="dynamic",                                                   303
                                    cascade="all, delete-orphan")     304
class ReferenceEcosystem(db.Model):                                   305
    """参照生态系统：标准化基准值，与图斑绑定。"""                    306
    __tablename__ = "reference_ecosystems"                            307
    id = db.Column(db.Integer, primary_key=True)                      308
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat          309
    ches.id"), nullable=False)                                        310
    coverage_benchmark = db.Column(db.Float, default=75.0)            311
    # 植被覆盖度基准 %                                                312
    shannon_benchmark = db.Column(db.Float, default=0.7)              313
    # Shannon-Wiener 基准                                             314
    dominant_species = db.Column(db.String(256),                      315
    nullable=True)  # 建群种                                         316
    soil_benchmark = db.Column(db.String(256),                        317
    nullable=True)     # 土壤基准                                     318
class Standard(db.Model):                                             319
    """地方标准 / 政策文件（政策与标准知识库）。"""                   320
    __tablename__ = "standards"                                       321
    id = db.Column(db.Integer, primary_key=True)                      322
    province = db.Column(db.String(32), nullable=False)               323
    # 省份                                                            324
    name = db.Column(db.String(256), nullable=False)                  325
    # 名称                                                            326
    doc_number = db.Column(db.String(128), nullable=True)             327
    # 标准编号 / 文号                                                 328
    publish_date = db.Column(db.String(32), nullable=True)            329
    implement_date = db.Column(db.String(32), nullable=True)          330
    # 类别：地方标准 / 政策文件                                       331
    category = db.Column(db.String(32), nullable=False,               332
    default="地方标准")                                               333
    summary = db.Column(db.Text, nullable=True)                       334
    # 特点概述                                                        335
    rules = db.relationship("StandardRule",                           336
    backref="standard",                                               337
                                    cascade="all, delete-orphan")     338
class StandardRule(db.Model):                                         339
    """结构化规则：将标准条文拆解为可执行规则。"""                    340
    __tablename__ = "standard_rules"                                  341
    id = db.Column(db.Integer, primary_key=True)                      342
    standard_id = db.Column(db.Integer,                               343
    db.ForeignKey("standards.id"), nullable=False)                    344
    item = db.Column(db.String(128), nullable=False)                  345
    # 判定项                                                          346
    operator = db.Column(db.String(8), nullable=False)                347
    # 比较符：> >= < <= ==                                            348
    threshold = db.Column(db.String(64), nullable=True)               349
    # 阈值                                                            350
```

```
    condition_type = db.Column(db.String(32),                    351
    default="必要条件")    # 必要条件 / 充分条件                    352
    region = db.Column(db.String(64), nullable=True)             353
    # 适用分区（宁夏分区差异化）                                    354
    note = db.Column(db.Text, nullable=True)                     355
class ConstraintCheck(db.Model):                                 356
    """限制性判定记录。"""                                         357
    __tablename__ = "constraint_checks"                          358
    id = db.Column(db.Integer, primary_key=True)                 359
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat     360
    ches.id"), nullable=False)                                   361
    policy_pass = db.Column(db.Boolean, nullable=True)           362
    # 政策约束                                                     363
    location_pass = db.Column(db.Boolean, nullable=True)         364
    # 区位约束                                                     365
    pollution_pass = db.Column(db.Boolean, nullable=True)        366
    # 土壤污染                                                     367
    geo_pass = db.Column(db.Boolean, nullable=True)              368
    # 地质灾害                                                     369
    farmland_pass = db.Column(db.Boolean, nullable=True)         370
    # 耕地属性                                                     371
    passed = db.Column(db.Boolean, nullable=True)                372
    # 总体是否通过                                                 373
    reason = db.Column(db.Text, nullable=True)                   374
    # 不通过原因                                                   375
    checked_at = db.Column(db.DateTime,                          376
    default=datetime.utcnow)                                     377
class MicroUnit(db.Model):                                       378
    """微地质单元：岩质边坡 / 碎石边坡 / 平台。"""                  379
    __tablename__ = "micro_units"                                380
    id = db.Column(db.Integer, primary_key=True)                 381
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat     382
    ches.id"), nullable=False)                                   383
    unit_type = db.Column(db.String(32), nullable=False)         384
    # 单元类型                                                     385
    area = db.Column(db.Float, nullable=True)                    386
    # 面积                                                         387
    ratio = db.Column(db.Float, nullable=True)                   388
    # 面积占比（自动计算）                                         389
    description = db.Column(db.Text, nullable=True)              390
    # 现场描述                                                     391
class Indicator(db.Model):                                       392
    """指标定义：指标树，含角色与启用单元类型。"""                  393
    __tablename__ = "indicators"                                 394
    id = db.Column(db.Integer, primary_key=True)                 395
    code = db.Column(db.String(32), unique=True,                 396
    nullable=False)    # 指标编码                                397
    name = db.Column(db.String(64), nullable=False)              398
    # 指标名称                                                     399
    level = db.Column(db.Integer, nullable=False)                400
```

```
    #  1/2/3 级                                                                    401
    parent_code = db.Column(db.String(32), nullable=True)                         402
    #  父指标编码                                                                  403
    #  角色：土壤污染指标同时作为限制性指标与评价指标                              404
    is_constraint = db.Column(db.Boolean, default=False)                          405
    is_evaluative = db.Column(db.Boolean, default=True)                           406
    unit = db.Column(db.String(32), nullable=True)                                407
    #  单位                                                                        408
    ideal_value = db.Column(db.String(64), nullable=True)                         409
    #  理想值（可配置）                                                            410
    ideal_type = db.Column(db.String(32), nullable=True)                          411
    #  benefit/cost/mid/discrete                                                  412
    #  启用该指标的微地质单元（逗号分隔）。岩质边坡不启用土壤指标、平台不启用地形指标  413
    enabled_units = db.Column(db.String(128), nullable=True)                      414
    description = db.Column(db.Text, nullable=True)                               415
    #  取值依据说明                                                                416
    def units_list(self):                                                         417
        return [s for s in (self.enabled_units or                                 418
    "").split(",") if s]                                                          419
class WeightScheme(db.Model):                                                     420
    """权重方案：按微地质单元类型分别保存。"""                                    421
    __tablename__ = "weight_schemes"                                              422
    id = db.Column(db.Integer, primary_key=True)                                  423
    unit_type = db.Column(db.String(32), nullable=False)                          424
    #  所属单元类型                                                                425
    name = db.Column(db.String(64), nullable=True)                                426
    #  方案名称                                                                    427
    level = db.Column(db.Integer, nullable=False,                                 428
    default=1)          #  1=一级指标  2=二级指标                                 429
    is_default = db.Column(db.Boolean, default=False)                             430
    #  是否默认（论文表 3）                                                        431
    created_at = db.Column(db.DateTime,                                           432
    default=datetime.utcnow)                                                      433
class WeightItem(db.Model):                                                       434
    """权重明细：方案内的各项权重。"""                                            435
    __tablename__ = "weight_items"                                                436
    id = db.Column(db.Integer, primary_key=True)                                  437
    scheme_id = db.Column(db.Integer,                                             438
    db.ForeignKey("weight_schemes.id"), nullable=False)                           439
    indicator_code = db.Column(db.String(32),                                     440
    nullable=False)         #  指标编码                                           441
    weight = db.Column(db.Float, nullable=False)                                  442
    #  权重值                                                                      443
class ExpertMatrix(db.Model):                                                     444
    """专家判断矩阵：多专家打分的原始矩阵。"""                                    445
    __tablename__ = "expert_matrices"                                             446
    id = db.Column(db.Integer, primary_key=True)                                  447
    scheme_id = db.Column(db.Integer,                                             448
    db.ForeignKey("weight_schemes.id"), nullable=True)                            449
    #  直接标注适用对象，便于无需先建权重方案即可录入专家矩阵                      450
```

```
    unit_type = db.Column(db.String(32), nullable=True)
    # 微地质单元类型
    l1_name = db.Column(db.String(64), nullable=True)
    # 所属一级指标（二级矩阵用）
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 矩阵对应指标层级
    matrix_json = db.Column(db.Text, nullable=False)
    # 判断矩阵（JSON 二维数组）
    cr = db.Column(db.Float, nullable=True)
    # 该专家矩阵的一致性比例
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class IndicatorData(db.Model):
    """指标实测数据：按微地质单元组织。"""
    __tablename__ = "indicator_data"
    id = db.Column(db.Integer, primary_key=True)
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_uni
    ts.id"), nullable=False, unique=True)
    soil_ratio = db.Column(db.Float, nullable=True)
    # 土石比
    thickness = db.Column(db.Float, nullable=True)
    # 土层厚度 cm
    ph = db.Column(db.Float, nullable=True)
    organic_matter = db.Column(db.Float, nullable=True)
    # 有机质 g/kg
    # 重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null
    heavy_metals = db.Column(db.Text, nullable=True)
    coverage = db.Column(db.Float, nullable=True)
    # 植被覆盖度 %
    shannon = db.Column(db.Float, nullable=True)
    # Shannon-Wiener 多样性指数
    slope = db.Column(db.Float, nullable=True)
    # 坡度 °
    aspect = db.Column(db.String(16), nullable=True)
    # 坡向：阳坡/阴阳坡/阴坡
    def heavy_metals_dict(self):
        return json.loads(self.heavy_metals) if
    self.heavy_metals else {}
class EvaluationResult(db.Model):
    """评价结果。"""
    __tablename__ = "evaluation_results"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    final_score = db.Column(db.Float, nullable=True)
    # 最终评价因子 F
    grade = db.Column(db.String(32), nullable=True)
    # 适宜性等级
```

```
    measures = db.Column(db.Text, nullable=True)
    # 推荐措施
    unit_scores = db.Column(db.Text, nullable=True)
    # 各单元得分 JSON
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class ScoreDetail(db.Model):
    """得分明细：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""
    __tablename__ = "score_details"
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
from wtforms.validators import DataRequired, Length
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES
class AHPSelectForm(FlaskForm):
    """选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
```

```
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """登录表单。支持「记住我」以在关闭浏览器后保持登录状态。"""
    username = StringField("用户名", validators=[DataRequired(), Length(max=64)])
    password = PasswordField("密码", validators=[DataRequired()])
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
# ==== app/utils/constants.py ====
# -*- coding: utf-8 -*-
"""评价指标常量表。
全部数值取自论文，作为系统计算的唯一基准：
- 论文 4.2.2 节：指标标准化理想极值与取值依据
- 论文 4.2.3 节 / 表 4：适宜性等级划分
- 论文表 3：按微地质单元分别确定的一级、二级指标权重
- 论文 4.2.1 节：平均随机一致性指标 RI
"""
# ------------------------------------------------------------------
# 一、微地质单元类型
# ------------------------------------------------------------------
# 论文 4.1 节："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡
# 三种地质微单元类型"
UNIT_TYPES = ("岩质边坡", "碎石边坡", "平台")
# 每个单元类型启用的指标类别（论文表 3 隐含：岩质边坡无土壤指标，平台无地形指标）
# 供数据录入界面与评价引擎判断"该单元是否需要某类指标"
UNIT_ENABLED_INDICATORS = {
    "岩质边坡": ("植被指标", "地形指标"),                    # 不设土壤指标
    "碎石边坡": ("土壤指标", "植被指标", "地形指标"),
    "平台": ("土壤指标", "植被指标"),                        # 不设地形指标
}
# ------------------------------------------------------------------
# 二、指标标准化理想极值（论文 4.2.2 节）
# ------------------------------------------------------------------
IDEAL_SOIL_RATIO = 0.8          # 土石比理想值
IDEAL_THICKNESS = 20.0          # 土层厚度理想值 （cm）
IDEAL_PH_RANGE = (5.0, 9.0)     # pH 理想区间（适中型指标）
IDEAL_ORGANIC = 30.0            # 有机质理想值
    （g/kg），全国第二次土壤普查养分等级划分标准二等最低值
IDEAL_COVERAGE = 75.0           # 植被覆盖度参照生态系统 （%）
IDEAL_SHANNON = 0.7             # Shannon-Wiener 多样性指数参照生态系统
# 重金属极值 （mg/kg），《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值
# 论文原文：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、530、400、45、220、600、800
HEAVY_METALS = ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu",
```

```
    "Zn")
HEAVY_METAL_LIMIT = {
    "Cd": 2.0, "Hg": 2.0, "Pb": 530.0, "Cr": 400.0,
    "As": 45.0, "Ni": 220.0, "Cu": 600.0, "Zn": 800.0,
}
# 污染指标下 8 种重金属"重要性相当，权重平均分配"（论文 4.2.1 节）
HEAVY_METAL_EQUAL_WEIGHT = 1.0 / 8.0
# 坡度分界，论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号
SLOPE_BREAK = 25.0
# 坡向离散赋值，论文 4.2.2 节："阳坡、阴阳坡、阴坡分别赋值 0.2、0.6、1"
ASPECT_SCORES = {"阳坡": 0.2, "阴阳坡": 0.6, "阴坡": 1.0}
ASPECT_TYPES = ("阳坡", "阴阳坡", "阴坡")
# pH 适中型标准化的容差边界（区间外线性衰减，避免得分突变）
PH_TOL_LOW = 4.0
PH_TOL_HIGH = 10.0
# ---------------------------------------------------------------------
    ------------------
# 三、适宜性等级划分（论文表 4）
# ---------------------------------------------------------------------
    ------------------
GRADE_LEVELS = (
    {"grade": "不适宜", "min": None, "max": 0.3,
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",
     "measure": "生态重建、人工辅助"},
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},
    {"grade": "适宜", "min": 0.7, "max": None,
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",
     "measure": "自然恢复"},
)
def grade_of(score):
    """按论文表 4 判定适宜性等级。"""
    for g in GRADE_LEVELS:
        lo, hi = g["min"], g["max"]
        if lo is None and score < hi:
            return g
        if hi is None and score >= lo:
            return g
        if lo is not None and hi is not None and lo <=
    score < hi:
            return g
    # score 恰好等于 0.7 时落入"适宜"
    return GRADE_LEVELS[-1]
# ---------------------------------------------------------------------
    ------------------
# 四、平均随机一致性指标 RI（论文 4.2.1 节给出 n=2，3，4，5，其余为常用扩展值）
# ---------------------------------------------------------------------
    ------------------
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:
```

```
    1.24,                                                                    651
                7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}                         652
# 一致性比例阈值（论文：当 CR<0.1 时，判断矩阵满足一致性要求）                  653
CR_THRESHOLD = 0.1                                                           654
# ------------------------------------------------------------------------   655
    ----------                                                               656
# 五、默认权重方案（论文表 3）                                                657
# ------------------------------------------------------------------------   658
    ----------                                                               659
# 论文 4.2.2 节："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，  660
# 分别计算评价指标权重"（论文摘要、图 2 流程与表 5 均作"碎石边坡"，           661
# 系统统一采用"碎石边坡"）。权重由 10 位专家打分后经 AHP 方法确定。           662
# 下表为论文表 3 所载的归一化权重，作为系统默认方案。                         663
DEFAULT_WEIGHTS = {                                                          664
    "岩质边坡": {                                                            665
        "l1": {"植被指标": 0.45, "地形指标": 0.55},                          666
        "l2": {                                                              667
            "植被指标": {"植被覆盖度": 0.67, "生物多样性": 0.33},            668
            "地形指标": {"坡度": 0.64, "坡向": 0.36},                        669
        },                                                                   670
    },                                                                       671
    "碎石边坡": {                                                            672
        "l1": {"土壤指标": 0.44, "植被指标": 0.31, "地形指标": 0.25},        673
        "l2": {                                                              674
            "土壤指标": {"土石比": 0.25, "土层厚度": 0.25, "pH": 0.17,       675
                         "有机质": 0.17, "污染指标": 0.17},                  676
            "植被指标": {"植被覆盖度": 0.64, "生物多样性": 0.36},            677
            "地形指标": {"坡度": 0.67, "坡向": 0.33},                        678
        },                                                                   679
    },                                                                       680
    "平台": {                                                                681
        "l1": {"土壤指标": 0.54, "植被指标": 0.46},                          682
        "l2": {                                                              683
            "土壤指标": {"土石比": 0.25, "土层厚度": 0.25, "pH": 0.17,       684
                         "有机质": 0.17, "污染指标": 0.17},                  685
            "植被指标": {"植被覆盖度": 0.61, "生物多样性": 0.39},            686
        },                                                                   687
    },                                                                       688
}                                                                            689
# ------------------------------------------------------------------------   690
    ----------                                                               691
# 六、限制性指标（论文表 2 + 4.1 节流程）                                     692
# ------------------------------------------------------------------------   693
    ----------                                                               694
# 论文摘要与 4.1 节：四项限制性指标；流程段另明确"是否为耕地"为约束条件       695
CONSTRAINT_ITEMS = ("政策", "区位约束", "土壤污染", "地质灾害", "耕地属性")   696
# ==== app/utils/schema.py ====                                              697
# -*- coding: utf-8 -*-                                                      698
"""轻量结构迁移。                                                            699
SQLite 下 `db.create_all()` 只建缺失的表，不会给已有表补列。                 700
```

```
本模块在启动时检查目标列是否存在，缺失则 ALTER TABLE 补上，
使旧数据库无需重建即可用上新字段。
"""
from sqlalchemy import text
# （表名， 列名， 列定义）
REQUIRED_COLUMNS = [
    # 遥感影像挂接
    ("mine_patches", "image_path", "VARCHAR(256)"),
    ("mine_patches", "image_note", "VARCHAR(256)"),
    ("mine_patches", "image_uploaded_at", "DATETIME"),
    # 指标维护
    ("indicators", "enabled_units", "VARCHAR(128)"),
    ("indicators", "description", "TEXT"),
    # 多专家矩阵
    ("expert_matrices", "unit_type", "VARCHAR(32)"),
    ("expert_matrices", "l1_name", "VARCHAR(64)"),
    ("expert_matrices", "cr", "FLOAT"),
]
def ensure_schema(db):
    """为已有表补齐缺失列。返回补列清单。"""
    added = []
    try:
        with db.engine.connect() as conn:
            for table, column, ddl in REQUIRED_COLUMNS:
                rows = conn.execute(text("PRAGMA
    table_info(%s)" % table)).fetchall()
                if not rows:
                    continue                                #
    表还不存在，create_all 会建
                existing = {r[1] for r in rows}
                if column in existing:
                    continue
                conn.execute(text(
                    "ALTER TABLE %s ADD COLUMN %s %s" %
    (table, column, ddl)))
                conn.commit()
                added.append("%s.%s" % (table, column))
    except Exception as e:                                  #
    迁移失败不应阻断启动
        print("[schema] 结构检查跳过：%s" % e)
    return added
# ==== app/utils/uploads.py ====
# -*- coding: utf-8 -*-
"""上传文件处理（遥感影像、批量导入）。"""
import os
import re
import uuid
from datetime import datetime
from werkzeug.utils import secure_filename
def _safe_ext(filename):
```

```
    return os.path.splitext(filename)[1].lower().lstrip(".")
def ext_allowed(filename, allowed):
    return _safe_ext(filename) in allowed
def save_upload(file_storage, subdir, allowed_ext,
    upload_root):
    """保存上传文件，返回相对 static 的路径（如 uploads/patches/xxx.png）。
    文件名重命名为「时间戳_随机串.扩展名」，避免中文名与重名问题。
    """
    if file_storage is None or not file_storage.filename:
        return None, "未选择文件"
    ext = _safe_ext(file_storage.filename)
    if ext not in allowed_ext:
        return None, "不支持的文件类型：.%s" % ext
    target_dir = os.path.join(upload_root, subdir)
    os.makedirs(target_dir, exist_ok=True)
    # 仅保留安全字符，并加时间戳与随机串防重名
    base = secure_filename(os.path.splitext(file_storage.fil
    ename)[0]) or "file"
    base = re.sub(r"[^A-Za-z0-9_\-]", "", base)[:40] or
    "file"
    name = "%s_%s.%s" % (datetime.now().strftime("%Y%m%d%H%M
    %S"),
                         uuid.uuid4().hex[:6], ext)
    abs_path = os.path.join(target_dir, name)
    file_storage.save(abs_path)
    rel = os.path.join("uploads", subdir,
    name).replace("\\", "/")
    return rel, None
def remove_upload(rel_path, upload_root):
    """删除已保存的上传文件（相对 static 的路径）。"""
    if not rel_path:
        return
    abs_path = os.path.join(upload_root,
    rel_path.replace("uploads/", "", 1))
    try:
        if os.path.isfile(abs_path):
            os.remove(abs_path)
    except OSError:
        pass
# ==== app/utils/charts.py ====
# -*- coding: utf-8 -*-
"""轻量图表。
用纯 SVG 输出，不依赖任何绘图库或前端脚本：
- 服务端渲染，页面与打印（PDF）中都能稳定显示
- 每个图元都带数值标签，黑白打印时仍可读（不单靠颜色区分）
- 单色方案，符合界面「一个视图一个强调色」的约定
"""
from html import escape
ACCENT = "#1f3a5f"
ACCENT_SOFT = "#7f93ad"
```

```
INK = "#23262b"
MUTED = "#6b7280"
LINE = "#dfe3e8"
#  灰度梯度：黑白打印时相邻扇区 / 条柱仍可区分
GRAYS = ["#1f3a5f", "#456188", "#6d86a8", "#96a9c4",
    "#bcc9da", "#dbe3ec"]
def _fmt(v, digits=4):
    try:
        return ("%%.%df" % digits) % float(v)
    except (TypeError, ValueError):
        return str(v)
def hbars(items, width=620, bar_h=22, gap=12, label_w=150,
    value_w=76):
    """水平条形图。items: [(标签, 数值, 显示文本)]"""
    if not items:
        return '<p class="muted">暂无数据</p>'
    n = len(items)
    height = n * (bar_h + gap) + 10
    max_v = max((abs(float(v)) for _, v, _ in items),
    default=1) or 1
    body_w = width - label_w - value_w
    parts = ['<svg viewBox="0 0 %d %d" width="100%%"
    height="%d" '
             'role="img" aria-label="条形图" '
             'style="max-width:%dpx;font-family:inherit;">'
    % (width, height, height, width)]
    for i, (label, value, disp) in enumerate(items):
        y = i * (bar_h + gap) + 5
        frac = (abs(float(value)) / max_v) if max_v else 0
        w = max(2.0, body_w * frac)
        fill = GRAYS[i % len(GRAYS)]
        parts.append(
            '<text x="%d" y="%d" text-anchor="end"
    font-size="12" fill="%s">%s</text>'
            % (label_w - 10, y + bar_h * 0.72, INK,
    escape(str(label))))
        parts.append(
            '<rect x="%d" y="%d" width="%.1f" height="%d"
    rx="3" fill="%s"/>'
            % (label_w, y, w, bar_h, fill))
        parts.append(
            '<text x="%d" y="%d" font-size="12" fill="%s" '
            'style="font-variant-numeric:tabular-nums;">%s</
    text>'
            % (label_w + w + 8, y + bar_h * 0.72, MUTED,
    escape(str(disp))))
    parts.append('</svg>')
    return "".join(parts)
def donut(items, size=210, thickness=34):
    """环形图（权重分布）。items: [(标签, 数值, 显示文本)]
```

```
图例中给出数值与占比，黑白打印时靠图例与灰度深浅区分。
"""
vals = [(l, float(v), d) for l, v, d in items if
float(v) > 0]
if not vals:
    return '<p class="muted">暂无数据</p>'
total = sum(v for _, v, _ in vals) or 1
r = (size - thickness) / 2
cx = cy = size / 2
import math
parts = ['<svg viewBox="0 0 %d %d" width="%d"
height="%d" role="img" '
         'aria-label="权重分布环形图">' % (size, size, size,
size)]
angle = -math.pi / 2
for i, (label, v, disp) in enumerate(vals):
    sweep = 2 * math.pi * (v / total)
    a0, a1 = angle, angle + sweep
    x0, y0 = cx + r * math.cos(a0), cy + r *
math.sin(a0)
    x1, y1 = cx + r * math.cos(a1), cy + r *
math.sin(a1)
    large = 1 if sweep > math.pi else 0
    parts.append(
        '<path d="M %.2f %.2f A %.2f %.2f 0 %d 1 %.2f
%.2f" '
        'fill="none" stroke="%s" stroke-width="%d"/>'
        % (x0, y0, r, r, large, x1, y1, GRAYS[i %
len(GRAYS)], thickness))
    angle = a1
parts.append('<text x="%d" y="%d" text-anchor="middle"
font-size="12" '
             'fill="%s">合计</text>' % (cx, cy - 2,
MUTED))
parts.append('<text x="%d" y="%d" text-anchor="middle"
font-size="15" '
             'fill="%s" style="font-variant-numeric:tabu
lar-nums;">%s</text>'
             % (cx, cy + 18, INK, _fmt(total, 4)))
parts.append('</svg>')
legend = ['<table style="width:auto;margin-top:10px;"><t
head><tr>'
          '<th scope="col">项目</th><th
scope="col">数值</th>'
          '<th scope="col">占比</th></tr></thead><tbody>']
for i, (label, v, disp) in enumerate(vals):
    legend.append(
        '<tr><td style="text-align:left;">'
        '<span style="display:inline-block;width:10px;he
ight:10px;'
```

```
                'border-radius:2px;background:%s;margin-right:6p
    x;"></span>%s</td>'
                '<td class="mono">%s</td><td
    class="mono">%.1f%%</td></tr>'
              % (GRAYS[i % len(GRAYS)], escape(str(label)),
    escape(str(disp)),
                 100.0 * v / total))
    legend.append('</tbody></table>')
    return ('<div style="display:flex;flex-wrap:wrap;gap:20p
    x;align-items:center;">'
              + "".join(parts) + "".join(legend) + '</div>')
# ==== app/utils/browser.py ====
# -*- coding: utf-8 -*-
"""HTML 转 PDF（报告导出）。
复用本机已安装的 Chrome/Edge 无头模式打印，避免引入 reportlab、
weasyprint 等额外依赖。生成过程使用系统临时目录中的 ASCII 文件名，
避免中文路径在 file:// URL 下出问题。
"""
import os
import shutil
import subprocess
import tempfile
import uuid
CANDIDATES = [
    r"C:\Program Files\Google\Chrome\Application\chrome.exe"
    ,
    r"C:\Program Files (x86)\Google\Chrome\Application\chrom
    e.exe",
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msed
    ge.exe",
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe
    ",
]
# 允许通过环境变量覆盖，便于在其他机器上指定浏览器
ENV_KEY = "DSH_BROWSER_PATH"
def browser_path():
    """定位可用的浏览器可执行文件，找不到返回 None。"""
    override = os.environ.get(ENV_KEY)
    if override and os.path.isfile(override):
        return override
    for p in CANDIDATES:
        if os.path.isfile(p):
            return p
    for name in ("chrome", "msedge", "chromium"):
        found = shutil.which(name)
        if found:
            return found
    return None
def html_to_pdf(html, timeout=90):
    """把 HTML 字符串渲染为 PDF，返回字节；失败返回 (None, 原因)。"""
```

```
    exe = browser_path()
    if exe is None:
        return None, ("未找到可用的浏览器（Chrome/Edge），无法生成 PDF。"
                      "可设置环境变量 %s 指向浏览器可执行文件。" % ENV_KEY)
    tmpdir = tempfile.mkdtemp(prefix="dshpdf_")
    tag = uuid.uuid4().hex[:8]
    html_path = os.path.join(tmpdir, "r_%s.html" % tag)
    pdf_path = os.path.join(tmpdir, "r_%s.pdf" % tag)
    try:
        with open(html_path, "w", encoding="utf-8") as f:
            f.write(html)
        cmd = [
            exe, "--headless=new", "--disable-gpu",
    "--no-sandbox",
            "--no-first-run", "--no-default-browser-check",
            "--no-pdf-header-footer",
            "--virtual-time-budget=4000",
            "--print-to-pdf=%s" % pdf_path,
            "file:///" + html_path.replace("\\", "/"),
        ]
        p = subprocess.run(cmd, capture_output=True,
    text=True,
                           timeout=timeout,
    encoding="utf-8", errors="replace")
        if not os.path.isfile(pdf_path) or
    os.path.getsize(pdf_path) < 800:
            return None, "浏览器未能生成 PDF（%s）" % (p.stderr or
    "").strip()[:200]
        with open(pdf_path, "rb") as f:
            return f.read(), None
    except subprocess.TimeoutExpired:
        return None, "生成 PDF 超时，请稍后重试"
    except Exception as e:
        return None, "生成 PDF 失败：%s" % e
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True)
# ==== app/services/ahp_service.py ====
# -*- coding: utf-8 -*-
"""AHP 权重引擎。
严格按论文 4.2.1 节实现：
- 式(2)(3)(4)：特征根法求权重（行元素乘积 → n 次方根 → 归一化）
- 式(5)(6)(7)：一致性检验（λmax、CI、CR）
- 论文要求 CR < 0.1；RI 按阶数查表
"""
import numpy as np
from app.utils.constants import RI_TABLE, CR_THRESHOLD
def matrix_from_upper(upper):
    """由判断矩阵上三角构造完整互反矩阵。
    参数
    ----
```

```
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
            A[j, i] = 1.0 / upper[idx]
            idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict：{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
```

```
    返回                                                                1051
    ----                                                                1052
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)          1053
    及对应取值。                                                        1054
    """                                                                 1055
    A = np.asarray(A, dtype=float)                                      1056
    r = weights_and_consistency(A)                                      1057
    w = r["weights"]                                                    1058
    n = A.shape[0]                                                      1059
    best = None                                                         1060
    for i in range(n):                                                  1061
        for j in range(i + 1, n):                                       1062
            expected = w[i] / w[j]                                      1063
            if expected == 0:                                           1064
                continue                                                1065
            deviation = abs(A[i, j] - expected) / expected              1066
            if best is None or deviation >                              1067
    best["deviation"]:                                                  1068
                best = {"i": i, "j": j, "value": A[i, j],               1069
                        "expected": expected, "deviation":              1070
    deviation}                                                          1071
    return best                                                         1072
def aggregate_matrices(matrices):                                       1073
    """多专家判断矩阵聚合：几何平均（保持互反性）。                     1074
    参数                                                                1075
    ----                                                                1076
    matrices : list[np.ndarray]，多个同阶判断矩阵。                     1077
    """                                                                 1078
    stack = np.stack([np.asarray(m, dtype=float) for m in               1079
    matrices])                                                          1080
    return np.prod(stack, axis=0) ** (1.0 / len(stack))                 1081
def judge_from_weights(weights):                                        1082
    """由权重向量构造完全一致判断矩阵并检验。                           1083
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量     1084
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。                          1085
    """                                                                 1086
    w = np.asarray(weights, dtype=float)                                1087
    n = w.shape[0]                                                      1088
    A = np.outer(w, 1.0 / w)                                            1089
    r = weights_and_consistency(A)                                      1090
    r["matrix"] = A                                                     1091
    return r                                                            1092
# ==== app/services/standardization.py ====                           1093
# -*- coding: utf-8 -*-                                               1094
"""指标标准化（的一部分）。                                           1095
按论文 4.2.2 节式(8) 实现：                                           1096
    x' = (x - x_min) / (x_max - x_min)                                  1097
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。           1098
所有理想极值取自 app.utils.constants，可配置。                        1099
"""                                                                     1100
```

```
from app.utils.constants import (
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,
    PH_TOL_LOW, PH_TOL_HIGH,
)
def std_benefit(value, ideal):
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, value / ideal))
def std_cost(value, extreme):
    """越小越优指标标准化：以极值为参考，线性折减至 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, 1.0 - value / extreme))
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,
    tol_hi=PH_TOL_HIGH):
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""
    if value is None:
        return None
    if lo <= value <= hi:
        return 1.0
    if value > hi:
        return max(0.0, (tol_hi - value) / (tol_hi - hi))
    return max(0.0, (value - tol_lo) / (lo - tol_lo))
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):
    """坡度标准化。
    论文 4.2.2 节：以 25° 为陡坡重要分界。25° 及以下记满分 1，
    超过 25° 线性折减，90° 记 0。
    """
    if value is None:
        return None
    if value <= break_point:
        return 1.0
    return max(0.0, (upper - value) / (upper - break_point))
def aspect_score(aspect):
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（论文 4.2.2 节）。"""
    if aspect is None:
        return None
    return ASPECT_SCORES.get(aspect)
def pollution_score(heavy_metals):
    """污染指标：8 种重金属等权（各 0.125）合成。
    论文注：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染（=1）处理。
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。
    """
    total = 0.0
    for metal, extreme in HEAVY_METAL_LIMIT.items():
        value = heavy_metals.get(metal)
```

```
        total += std_cost(value, extreme) if value is not None else 1.0
    return total / len(HEAVY_METAL_LIMIT)
def standardize_unit(unit_type, data, weights=None):
    """对单个微地质单元的全部指标做标准化。
    参数
    ----
    unit_type : str，岩质边坡 / 碎石边坡 / 平台
    data : dict，实测数据，字段见接口契约：
        soil_ratio, thickness, ph, organic_matter,
    heavy_metals(dict),
        coverage, shannon, slope, aspect
    weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIGHTS 结构），
            未提供时用默认值。
    返回
    ----
    dict：{ 一级指标： { 二级指标： （实测值， 标准化值， 权重， 贡献） } }
    标准化值可能为 None，表示该指标不适用或方法待定。
    """
    from app.utils.constants import DEFAULT_WEIGHTS
    if weights is None:
        weights = DEFAULT_WEIGHTS[unit_type]
    l1_enabled = weights["l1"].keys()
    l2w = weights["l2"]
    result = {}
    # ---- 土壤指标 ----
    if "土壤指标" in l1_enabled:
        result["土壤指标"] = {
            "土石比": (data.get("soil_ratio"),
                       std_benefit(data.get("soil_ratio"),
    IDEAL_SOIL_RATIO),
                       l2w["土壤指标"]["土石比"]),
            "土层厚度": (data.get("thickness"),
                         std_benefit(data.get("thickness"),
    IDEAL_THICKNESS),
                         l2w["土壤指标"]["土层厚度"]),
            "pH": (data.get("ph"),
                   std_mid_ph(data.get("ph"),
    *IDEAL_PH_RANGE),
                   l2w["土壤指标"]["pH"]),
            "有机质": (data.get("organic_matter"),
                       std_benefit(data.get("organic_matter")
    , IDEAL_ORGANIC),
                       l2w["土壤指标"]["有机质"]),
            "污染指标": (data.get("heavy_metals") or {},
                         pollution_score(data.get("heavy_meta
    ls") or {}),
                         l2w["土壤指标"]["污染指标"]),
        }
```

```
    # ---- 植被指标 ----
    if "植被指标" in l1_enabled:
        result["植被指标"] = {
            "植被覆盖度": (data.get("coverage"),
                           std_benefit(data.get("coverage"),
    IDEAL_COVERAGE),
                           l2w["植被指标"]["植被覆盖度"]),
            "生物多样性": (data.get("shannon"),
                           std_benefit(data.get("shannon"),
    IDEAL_SHANNON),
                           l2w["植被指标"]["生物多样性"]),
        }
    # ---- 地形指标 ----
    if "地形指标" in l1_enabled:
        result["地形指标"] = {
            "坡度": (data.get("slope"),
                     std_slope(data.get("slope")),
                     l2w["地形指标"]["坡度"]),
            "坡向": (data.get("aspect"),
                     aspect_score(data.get("aspect")),
                     l2w["地形指标"]["坡向"]),
        }
    return result
# ==== app/services/weight_service.py ====
# -*- coding: utf-8 -*-
"""权重方案的存取与生效权重解析。
系统有两套权重来源：
1. 论文表 3 给出的默认权重（app.utils.constants.DEFAULT_WEIGHTS），
   是**权威取值来源**，案例复算与报告结论以此为准；
2. 通过层次分析法在本系统中实际算出的权重，保存在
   weight_schemes / weight_items 两张表中，可标记为「生效」。
本模块负责把这两者合成为计算引擎直接可用的嵌套权重结构：
以论文表 3 为底，用生效方案中已保存的条目覆盖。
没有任何生效方案时，即完全使用论文表 3 的权重。
这样既保证默认行为与论文一致，又使 AHP 计算结果真正参与评价，
而不是算完即弃。
"""
import copy
from app.models import db, WeightScheme, WeightItem
from app.utils.constants import DEFAULT_WEIGHTS
from app.services.indicator_service import TREE
# 方案名称：每个微地质单元类型固定一个可生效的方案
ACTIVE_NAME = "自定义权重方案（AHP 计算结果）"
def _code_maps():
    """由指标树导出 编码 → 名称 的映射，避免名称在两处各写一遍。"""
    l1, l2 = {}, {}
    for row in TREE:
        code, name, level, parent = row[0], row[1], row[2],
    row[3]
        if level == 1:
```

```
                l1[code] = name                                                  1251
            elif level == 2:                                                     1252
                l2[code] = (l1.get(parent, parent), name)                        1253
    return l1, l2                                                                1254
def effective_weights(unit_type):                                                1255
    """返回该单元类型当前生效的权重。                                                1256
    返回 (weights, source_label, scheme)                                         1257
        weights       嵌套权重结构，可直接交给标准化与评价引擎                          1258
        source_label  权重来源说明，用于界面与报告标注                                1259
        scheme        生效的权重方案对象，无自定义方案时为 None                       1260
    """                                                                          1261
    base = DEFAULT_WEIGHTS.get(unit_type)                                        1262
    if base is None:                                                             1263
        return None, "未定义该微地质单元类型的权重", None                              1264
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ                     1265
    e,                                                                           1266
                                                                                 1267
    name=ACTIVE_NAME).first()                                                    1268
    if scheme is None:                                                           1269
        return copy.deepcopy(base), "论文表 3 默认权重", None                        1270
    merged = copy.deepcopy(base)                                                 1271
    l1_map, l2_map = _code_maps()                                                1272
    applied = 0                                                                  1273
    for item in WeightItem.query.filter_by(scheme_id=scheme.                     1274
    id).all():                                                                   1275
        code, w = item.indicator_code, item.weight                               1276
        if w is None or w <= 0:                                                  1277
            continue                                                             1278
        if code in l1_map and l1_map[code] in merged["l1"]:                      1279
            merged["l1"][l1_map[code]] = w                                       1280
            applied += 1                                                         1281
        elif code in l2_map:                                                     1282
            l1_name, l2_name = l2_map[code]                                      1283
            if l1_name in merged["l2"] and l2_name in                            1284
    merged["l2"][l1_name]:                                                       1285
                merged["l2"][l1_name][l2_name] = w                               1286
                applied += 1                                                     1287
    if applied == 0:                                                             1288
        return copy.deepcopy(base), "论文表 3 默认权重", None                        1289
    return merged, "自定义方案（%d 项覆盖论文表 3）" % applied, scheme                 1290
def save_weights(unit_type, code_weight_pairs, note=None):                       1291
    """把一组「指标编码 → 权重」写入生效方案（已存在则更新对应条目）。                    1292
    仅覆盖本次传入的条目，其余条目保留，便于分层次（一级 / 二级）                        1293
    分次录入而无需一次填完。                                                       1294
    """                                                                          1295
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ                     1296
    e,                                                                           1297
                                                                                 1298
    name=ACTIVE_NAME).first()                                                    1299
    if scheme is None:                                                           1300
```

```
            scheme = WeightScheme(unit_type=unit_type,
        name=ACTIVE_NAME,
                                  level=1, is_default=True)
            db.session.add(scheme)
            db.session.flush()
        if note:
            scheme.level = scheme.level or 1
        if not scheme.is_default:
            scheme.is_default = True
        written = 0
        for code, w in code_weight_pairs:
            if w is None or w <= 0:
                continue
            item = WeightItem.query.filter_by(scheme_id=scheme.i
        d,

        indicator_code=code).first()
            if item is None:
                item = WeightItem(scheme_id=scheme.id,
        indicator_code=code)
                db.session.add(item)
            item.weight = float(w)
            written += 1
        db.session.commit()
        return scheme, written
def reset_weights(unit_type):
        """删除该单元类型的自定义方案，恢复为论文表 3 的默认权重。"""
        schemes = WeightScheme.query.filter_by(unit_type=unit_ty
        pe,

        name=ACTIVE_NAME).all()
        n = 0
        for s in schemes:
            WeightItem.query.filter_by(scheme_id=s.id).delete(
                synchronize_session=False)
            db.session.delete(s)
            n += 1
        db.session.commit()
        return n
def scheme_items(unit_type):
        """返回该单元类型已保存的自定义权重条目 {指标编码: 权重}。"""
        scheme = WeightScheme.query.filter_by(unit_type=unit_typ
        e,

        name=ACTIVE_NAME).first()
        if scheme is None:
            return {}, None
        items = {i.indicator_code: i.weight
                 for i in WeightItem.query.filter_by(scheme_id=s
        cheme.id).all()}
```

```
    return items, scheme
def summary():
    """各单元类型的权重来源概览，供权重首页展示。"""
    rows = []
    for unit_type in DEFAULT_WEIGHTS:
        weights, label, scheme = effective_weights(unit_type
    )
        rows.append({
            "unit_type": unit_type,
            "label": label,
            "custom": scheme is not None,
        })
    return rows
# ==== app/services/evaluation_service.py ====
# -*- coding: utf-8 -*-
"""适宜性评价引擎。
按论文式（9） 实现：
- 单元级：F' = Σ（二级指标权重 × 该二级指标下三级指标标准化值）
- 矿山级：F = Σ（Si × F'i），Si 为各微地质单元面积占比
- 等级判定按论文表 4。
"""
from app.utils.constants import DEFAULT_WEIGHTS, grade_of
from app.services import standardization as std
def score_unit(unit_type, data, weights=None):
    """计算单个微地质单元的评价因子 F'。
    返回
    ----
    dict：{ l1_scores: {一级指标: 得分}, unit_score: F', details:
    [...] }
    若某一级指标下存在"不适用"（标准化值为 None）的二级指标，
    则按其余项权重归一化后计算，保证单元得分始终有定义。
    """
    if weights is None:
        weights = DEFAULT_WEIGHTS[unit_type]
    std_result = std.standardize_unit(unit_type, data,
    weights)
    l1_weights = weights["l1"]
    l1_scores = {}
    details = []
    for l1_name, items in std_result.items():
        wsum = 0.0
        acc = 0.0
        for l2_name, (raw, std_value, w2) in items.items():
            if std_value is None:
                # 不适用项：不计入
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw":
    raw,
                    "std_value": None, "weight": w2,
    "contribution": None,
```

```
                    })
                    continue
                contribution = std_value * w2
                acc += contribution
                wsum += w2
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw": raw,
                    "std_value": std_value, "weight": w2,
                    "contribution": contribution,
                })
            l1_scores[l1_name] = acc / wsum if wsum > 0 else 0.0
        # 一级指标权重按实际参与计算的指标归一化。
        # 论文表 3 的权重本身已归一（和为 1），此处为恒等变换；
        # 若某层指标缺失或自定义权重不完整，可避免单元得分被整体压低。
        lw_sum = sum(l1_weights[k] for k in l1_scores if k in
        l1_weights)
        if lw_sum > 0:
            unit_score = sum(l1_weights[k] * l1_scores[k]
                             for k in l1_scores if k in
        l1_weights) / lw_sum
        else:
            unit_score = 0.0
        return {"l1_scores": l1_scores, "unit_score":
        unit_score, "details": details}
def score_mine(units, weights_map=None):
    """计算整个矿山的评价因子 F 与适宜性等级。
    参数
    ----
    units : list[dict]，每个元素：
        {unit_type, ratio, data}
        ratio 为该单元面积占比（0~1），data 为实测数据 dict。
    weights_map : dict，{单元类型: 权重结构}。未提供的单元类型
        使用论文表 3 的默认权重。权重可来自 AHP 计算并经
        weight_service 保存生效，从而真正参与评价。
    返回
    ----
    dict:{ final_score, grade, grade_info, unit_results }
    """
    weights_map = weights_map or {}
    total = 0.0
    unit_results = []
    for u in units:
        r = score_unit(u["unit_type"], u["data"],
                       weights_map.get(u["unit_type"]))
        weighted = u["ratio"] * r["unit_score"]
        total += weighted
        unit_results.append({
            "unit_type": u["unit_type"],
            "ratio": u["ratio"],
            "unit_score": r["unit_score"],
```

```
                "weighted": weighted,
                "l1_scores": r["l1_scores"],
                "details": r["details"],
            })
    grade_info = grade_of(total)
    return {
        "final_score": total,
        "grade": grade_info["grade"],
        "grade_info": grade_info,
        "unit_results": unit_results,
    }
# ==== app/services/report_service.py ====
# -*- coding: utf-8 -*-
"""报告生成：生成 Word 评价报告。
依赖 python-docx。报告结构：
标题 → 图斑信息 → 评价结论 → 各微地质单元得分 → 完整得分明细 → 生成时间。
"""
import io
from datetime import datetime
from docx import Document
from docx.shared import Pt, Cm
from docx.oxml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
def _set_run(run, text=None, size=12, bold=False):
    if text is not None:
        run.text = text
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    # 中文字体
    run._element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")
    return run
def _para(doc, text, size=12, bold=False, align=None,
    space_after=6):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    _set_run(p.add_run(), text, size, bold)
    return p
def _heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for run in h.runs:
        run.font.name = "Times New Roman"
        run._element.rPr.rFonts.set(qn("w:eastAsia"), "黑体")
    return h
def _table(doc, headers, rows, widths=None):
    t = doc.add_table(rows=1, cols=len(headers))
    t.style = "Table Grid"
    hdr = t.rows[0].cells
```

```
import sys
sys.stdout.reconfigure(encoding="utf-8")
import os
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app.services import evaluation_service as ev
from app.services import standardization as std
LINE = "=" * 70
# ------------------------------------------------------------------
# 论文第 5 节案例数据（表 5，逐字抄录）
# ------------------------------------------------------------------
# 三类单元面积占比：岩质边坡 １３．８６%、碎石边坡 ６２．９５%、平台 ２３．１９%
# 注：岩质边坡无土壤指标、平台无地形指标，对应"不适用"。
case = [
    {
        "unit_type": "岩质边坡",
        "ratio": 0.1386,
        "data": {
            "coverage": 68.88, "shannon": 0.00,
            "slope": 63.33, "aspect": "阴阳坡",
        },
    },
    {
        "unit_type": "碎石边坡",
        "ratio": 0.6295,
        "data": {
            "soil_ratio": 0.34, "thickness": 10, "ph":
    9.24, "organic_matter": 4.89,
            "heavy_metals": {"Hg": 0.05, "As": 5.17, "Ni":
    12.60,
                             "Cu": 11.70, "Cd": 0.04, "Pb":
    6.50},
            "coverage": 87.92, "shannon": 0.60,
            "slope": 28.75, "aspect": "阳坡",
        },
    },
    {
        "unit_type": "平台",
        "ratio": 0.2319,
        "data": {
            "soil_ratio": 0.56, "thickness": 30, "ph":
    8.72, "organic_matter": 10.92,
            "heavy_metals": {"Hg": 0.02, "As": 3.13, "Ni":
    12.33,
```

```
                                        "Cu": 11.14, "Cd": 0.09, "Pb": 7569
    9.98},                                                                7570
                "coverage": 58.42, "shannon": 0.72,                       7571
            },                                                            7572
    },                                                                    7573
]                                                                         7574
# ------------------------------------------------------------------------7575
    ------------------                                                    7576
# 逐单元输出标准化与得分                                                  7577
# ------------------------------------------------------------------------7578
    ------------------                                                    7579
print(LINE)                                                               7580
print("玉田县案例 · 评价引擎完整计算")                                   7581
print(LINE)                                                               7582
for u in case:                                                            7583
    r = ev.score_unit(u["unit_type"], u["data"])                          7584
    print()                                                               7585
    print("  【%s】  (面积占比 %.2f%%)" % (u["unit_type"],                7586
    u["ratio"] * 100))                                                    7587
    print("  " + "-" * 60)                                               7588
    for l1, score in r["l1_scores"].items():                              7589
        print("    %s 得分 = %.4f" % (l1, score))                        7590
    print("    单元评价因子 F' = %.4f" % r["unit_score"])                7591
print()                                                                   7592
print(LINE)                                                               7593
print("面积加权合成（式(9)）")                                           7594
print(LINE)                                                               7595
res = ev.score_mine(case)                                                 7596
total = res["final_score"]                                                7597
print()                                                                   7598
for ur in res["unit_results"]:                                            7599
    print("  %s  %.2f%% × %.4f = %.4f"                                   7600
          % (ur["unit_type"], ur["ratio"] * 100,                          7601
    ur["unit_score"], ur["weighted"]))                                    7602
print("  " + "-" * 60)                                                   7603
print("  最终评价因子 F = %.4f" % total)                                 7604
print()                                                                   7605
print("  适宜性等级：%s" % res["grade"])                                 7606
print("  判定依据  ：%s" % res["grade_info"]["meaning"])                 7607
print("  推荐措施  ：%s" % res["grade_info"]["measure"])                 7608
print()                                                                   7609
print(LINE)                                                               7610
print("结论")                                                            7611
print(LINE)                                                               7612
print("  - 评价引擎按论文式(8)(9) 与表 3 权重完整跑通")                  7613
print("  - 坡度标准化采用论文 25° 分界的线性折减默认值（见 standardization.py）")  7614
print("  - 最终 F ≈ %.2f，判定为「%s」，与论文「最终适宜自然恢复」的结论一致"  7615
      % (total, res["grade"]))                                            7616
print("  - 论文表 5 所载最终得分为 0.76；差异集中在坡度、坡向的标准化口径，")  7617
print("    该口径在 standardization.std_slope / aspect_score             7618
```

```
    中集中定义、可配置"）
print(LINE)
#  ==== 测试/test_web_ahp.py ====
#  -*- coding: utf-8 -*-
"""AHP 网页界面自检：走通选择→录入→计算 全流程。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import os
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file
    __)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
#  测试配置：关闭 CSRF 以简化测试客户端（真实运行仍开启 CSRF）
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///:memory:"
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
#  建立表并创建测试用户，登录后再访问业务页面
with app.app_context():
    from app.models import db, User
    from werkzeug.security import generate_password_hash
    db.create_all()
    if not User.query.filter_by(username="admin").first():
        db.session.add(User(username="admin",
                            password_hash=generate_password_
    hash("admin123"),
                            role="admin"))
        db.session.commit()
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  首页与 AHP 入口")
print(LINE)
r = client.get("/")
print("  首页状态码:", r.status_code, "（应 200）")
assert r.status_code == 200
r = client.get("/ahp/")
print("  AHP 入口状态码:", r.status_code, "（应 200）")
assert r.status_code == 200
assert "微地质单元类型" in r.get_data(as_text=True)
print()
print(LINE)
```

```
print("测试 2  选择碎石边坡一级指标 → 跳转矩阵页")
print(LINE)
r = client.post("/ahp/", data={
    "unit_type": "碎石边坡", "level": "1", "submit": "生成判断矩阵",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转后页面含「判断矩阵录入」:", "判断矩阵录入" in body)
print("  含三个一级指标（土壤/植被/地形）:", all(n in body for n in
    ("土壤指标", "植被指标", "地形指标")))
assert "判断矩阵录入" in body
assert all(n in body for n in ("土壤指标", "植被指标", "地形指标"))
print()
print(LINE)
print("测试 3  提交论文表 3 一致矩阵 → 还原权重")
print(LINE)
# 碎石边坡一级权重 (0.44, 0.31, 0.25) 对应的完全一致矩阵上三角：
# a12 = 0.44/0.31 = 1.4194, a13 = 0.44/0.25 = 1.76, a23 =
    0.31/0.25 = 1.24
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  结果页含「计算结果」:", "计算结果" in body)
print("  含权重 0.44:", "0.44" in body, " 含 0.31:", "0.31" in
    body, " 含 0.25:", "0.25" in body)
print("  含一致性检验通过:", "通过" in body)
assert "计算结果" in body
assert "0.44" in body and "0.31" in body and "0.25" in body
print()
print(LINE)
print("测试 4  不一致矩阵应提示不通过并定位")
print(LINE)
# 明显不一致：a12=9, a13=1, a23=9
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "9", "m_0_2": "1", "m_1_2": "9",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  含「不通过」:", "不通过" in body)
print("  含「建议优先调整」:", "建议优先调整" in body)
assert "不通过" in body
assert "建议优先调整" in body
print()
print(LINE)
print("网页界面全部测试通过 √")
print(LINE)
# ==== 测试/test_web_modules.py ====
# -*- coding: utf-8 -*-
"""模块自检：登录、项目档案、微地质单元、标准库、限制性判定。"""
import os
import sys
```

```
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
print(LINE)
print("测试 1  未登录应被重定向到登录页")
print(LINE)
r = client.get("/project/")
print("  /project/ 状态码:", r.status_code, " 重定向到:",
    r.headers.get("Location"))
assert r.status_code == 302 and "login" in
    r.headers.get("Location", "")
print()
print(LINE)
print("测试 2  登录")
print(LINE)
r = client.post("/login", data={"username": "admin",
    "password": "admin123"},
                follow_redirects=True)
body = r.get_data(as_text=True)
print("  登录后含「系统功能」:", "系统功能" in body)
print("  含用户名 admin:", "admin" in body)
assert "系统功能" in body
print()
print(LINE)
print("测试 3  项目与矿山档案")
print(LINE)
r = client.get("/project/")
body = r.get_data(as_text=True)
print("  项目列表状态码:", r.status_code)
print("  含案例项目:", "玉田县典型历史遗留废弃矿山案例" in body)
assert "玉田县典型历史遗留废弃矿山案例" in body
# 新建项目
client.post("/project/add", data={"name": "测试项目A", "code":
    "T-001"})
r = client.get("/project/")
```

```
print("   新建项目后含「测试项目A」:", "测试项目A" in r.get_data(as_text=True
    ))
assert "测试项目A" in r.get_data(as_text=True)
#  清理测试数据，避免污染演示库
with app.app_context():
    from app.models import db, Project
    for p in Project.query.filter_by(name="测试项目A").all():
        db.session.delete(p)
    db.session.commit()
print("   已清理测试项目")
print()
print(LINE)
print("测试 4   微地质单元（面积占比自动计算）")
print(LINE)
from app.models import MinePatch
with app.app_context():
    patch = MinePatch.query.filter_by(name="玉田县孤树镇白云岩采石场").f
    irst()
    pid = patch.id
r = client.get("/unit/%d" % pid)
body = r.get_data(as_text=True)
print("   单元列表含三类单元:", all(t in body for t in ("岩质边坡",
    "碎石边坡", "平台")))
print("   含占比合计:", "占比合计" in body)
assert "岩质边坡" in body and "占比合计" in body
print()
print(LINE)
print("测试 5   标准知识库与省际口径比对")
print(LINE)
r = client.get("/standard/")
body = r.get_data(as_text=True)
print("   含辽宁/湖南/新疆标准:", all(p in body for p in ("辽宁", "湖南",
    "新疆")))
assert "辽宁" in body
r = client.get("/standard/compare")
body = r.get_data(as_text=True)
print("   口径比对页含「植被覆盖度」分组:", "植被覆盖度" in body)
print("   含多省份并列:", "湖南" in body and "新疆" in body)
assert "植被覆盖度" in body
print()
print(LINE)
print("测试 6   限制性指标判定（一票否决）")
print(LINE)
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("   含五项条件:", all(x in body for x in ("政策约束", "区位约束",
    "土壤污染", "地质灾害", "耕地属性")))
assert all(x in body for x in ("政策约束", "区位约束", "土壤污染",
    "地质灾害", "耕地属性"))
#  全部通过
```

```
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  全通过后显示「通过，可进入定量评价」:", "可进入定量评价" in body)
assert "可进入定量评价" in body
# 一项不通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "0",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  一项不通过后显示「不通过」:", "不通过" in body)
print("  含失败项「区位约束」:", "区位约束" in body)
assert "不通过" in body
# 恢复为全部通过，避免把演示数据留在「不通过」状态影响后续步骤
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
print("  已恢复为全部通过:", "可进入定量评价" in r.get_data(as_text=True))
print()
print(LINE)
print("全部测试通过 √")
print(LINE)
# ==== 测试/test_web_evaluate.py ====
# -*- coding: utf-8 -*-
"""端到端自检：录数据 → 执行评价 → 查看结果。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
# 与真实配置共用同一个 app.db，但关闭 CSRF 便于测试客户端
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
```

```
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  指标数据列表")
print(LINE)
r = client.get("/indicator/")
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含案例图斑：", "玉田县孤树镇白云岩采石场" in body)
print("  含三个单元：", all(n in body for n in ("岩质边坡", "碎石边坡",
    "平台")))
assert r.status_code == 200
assert "岩质边坡" in body and "碎石边坡" in body and "平台" in body
print()
print(LINE)
print("测试 2  指标数据录入页（按单元类型显示字段）")
print(LINE)
from app.models import MicroUnit, MinePatch
with app.app_context():
    unit_rock = MicroUnit.query.filter_by(unit_type="岩质边坡").
    first()
    unit_gravel = MicroUnit.query.filter_by(unit_type="碎石边坡"
    ).first()
r = client.get("/indicator/%d" % unit_rock.id)
b_rock = r.get_data(as_text=True)
print("  岩质边坡页：无「土石比」字段：", "土石比" not in b_rock, "（应 True）")
print("  岩质边坡页：含「植被覆盖度」：", "植被覆盖度" in b_rock)
assert "土石比" not in b_rock and "植被覆盖度" in b_rock
r = client.get("/indicator/%d" % unit_gravel.id)
b_gravel = r.get_data(as_text=True)
print("  碎石边坡页：含「土石比」字段：", "土石比" in b_gravel)
print("  碎石边坡页：含 8 种重金属：", all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn")))
assert "土石比" in b_gravel and all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn"))
print()
print(LINE)
print("测试 3  执行评价")
print(LINE)
with app.app_context():
    import seed_demo
    patch = seed_demo.demo_patch()
r = client.post("/evaluate/%d" % patch.id, data={},
    follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转至结果页：", "适宜性评价结果" in body)
```

```
print("   含最终评价因子 F:", "最终评价因子 F" in body)
print("   含等级「适宜」:", "适宜" in body)
print("   含推荐措施「自然恢复」:", "自然恢复" in body)
assert "最终评价因子 F" in body and "自然恢复" in body
print()
print(LINE)
print("测试 4  结果持久化")
print(LINE)
from app.models import EvaluationResult
with app.app_context():
    res = EvaluationResult.query.order_by(EvaluationResult.id.desc()).first()
    print("   最终得分 F =", res.final_score)
    print("   等级 =", res.grade)
    assert res.grade == "适宜"
    assert 0.7 <= res.final_score < 1.0
print()
print(LINE)
print("端到端全部测试通过 √")
print(LINE)
# ==== 测试/test_web_report.py ====
# -*- coding: utf-8 -*-
"""报告输出自检：得分明细页 + Word 导出。"""
import os
import sys
import io
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
from app.models import MinePatch, EvaluationResult
with app.app_context():
    import seed_demo
```

```
    patch = seed_demo.demo_patch()
    pid = patch.id
#  报告取自已保存的评价记录，因此先按网页路径真实执行一次评价
from app.views.evaluate import load_patch_input,
    save_evaluation
with app.app_context():
    units = load_patch_input(pid)
    rec = save_evaluation(pid, units)
    rid = rec.id
    print("前置：已执行评价并保存记录 id=%d，F=%.4f（%s）"
          % (rid, rec.final_score, rec.grade))
    from app.models import ScoreDetail
    n_detail = ScoreDetail.query.filter_by(result_id=rid).co
    unt()
    print("         逐层得分明细已落库：%d 条" % n_detail)
print()
print(LINE)
print("测试 1  报告列表")
print(LINE)
r = client.get("/report/")
print("  状态码：", r.status_code)
body = r.get_data(as_text=True)
print("  含「所属项目」列：", "所属项目" in body)
assert r.status_code == 200
print()
print(LINE)
print("测试 2  得分明细页")
print(LINE)
r = client.get("/report/%d" % pid)
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含「得分明细」：", "得分明细" in body)
print("  含「最终评价因子 F」：", "最终评价因子 F" in body)
print("  含二级指标明细（植被覆盖度 / 坡度 / 坡向）：", all(n in body for n in
    ("植被覆盖度", "坡度", "坡向")))
print("  含权重 / 标准化值列：", "标准化值" in body and "贡献" in body)
assert r.status_code == 200
assert "植被覆盖度" in body and "标准化值" in body
print()
print(LINE)
print("测试 3  Word 报告导出")
print(LINE)
r = client.get("/report/%d/word" % pid)
print("  状态码：", r.status_code)
print("  Content-Type:", r.headers.get("Content-Type",
    "")[:60])
data = r.data
print("  文件大小：", len(data), "字节")
print("  是否为有效 docx（PK 头）：", data[:2] == b"PK")
assert r.status_code == 200 and data[:2] == b"PK"
```

```
# 用 python-docx 打开验证内容
import docx
d = docx.Document(io.BytesIO(data))
text = "\n".join(p.text for p in d.paragraphs)
print("  含报告标题：", "自然恢复适宜性评价报告" in text)
print("  含评价结论章节：", "评价结论" in text)
print("  含得分明细章节：", "得分明细" in text)
print("  段落数：", len(d.paragraphs), " 表格数：", len(d.tables))
assert "自然恢复适宜性评价报告" in text and len(d.tables) >= 3
print()
print(LINE)
print("报告输出全部测试通过 √")
print(LINE)
# ==== 测试/test_ui_fixes.py ====
# -*- coding: utf-8 -*-
"""界面修复自检：「记住我」与各模块删除功能。"""
import os
import re
import sys
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
sys.stdout.reconfigure(encoding="utf-8")
LINE = "=" * 70
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
    REMEMBER_COOKIE_DURATION = __import__("datetime").timedelta(days=30)
app = create_app(TestConfig)
client = app.test_client()
# ------------------------------------------------------------------
print(LINE)
print("一、「记住我」")
print(LINE)
html = client.get("/login").get_data(as_text=True)
# 1） 复选框必须有可点击关联的 label（这是原先失效的根因）
wrapped = re.search(r'<label[^>]*class="checkbox"[^>]*>\s*<input[^>]*name="remember"',
                    html) is not None
print("  %s 「记住我」文字与复选框包在同一 label 内" % ("[符合]" if wrapped
```

```
        else "[不符合]"))
assert wrapped, "复选框未与文字关联，点击文字不会勾选"
has_for = 'for="remember"' in html
print("  %s label 带 for 属性指向复选框" % ("[符合]" if has_for else
        "[不符合]"))
# 2) 勾选后应下发带过期时间的 remember_token
c = app.test_client()
r = c.post("/login", data={"username": "admin", "password":
        "admin123",
                                   "remember": "y"},
        follow_redirects=False)
cookies = r.headers.getlist("Set-Cookie")
rt = [h for h in cookies if h.startswith("remember_token=")]
print("  %s 勾选后下发 remember_token" % ("[符合]" if rt else
        "[不符合]"))
assert rt, "未下发 remember_token"
exp = re.search(r"Expires=([^;]+)", rt[0])
print("  %s 该 cookie 带过期时间：%s"
        % ("[符合]" if exp else "[不符合]", exp.group(1) if exp
        else "无（会话 cookie）"))
assert exp, "remember_token 无过期时间，关浏览器即失效"
# 3) 不勾选时不应下发 remember_token
c2 = app.test_client()
r2 = c2.post("/login", data={"username": "admin",
        "password": "admin123"},
                   follow_redirects=False)
rt2 = [h for h in r2.headers.getlist("Set-Cookie") if
        h.startswith("remember_token=")]
print("  %s 未勾选时不下发 remember_token" % ("[符合]" if not rt2
        else "[不符合]"))
assert not rt2
# 4) 携带 remember_token 的新会话应自动登录
c3 = app.test_client()
c3.set_cookie("remember_token",
                    rt[0].split(";")[0].split("=", 1)[1],
        domain="localhost")
r3 = c3.get("/", follow_redirects=False)
print("  %s 仅凭 remember_token 访问首页可自动登录（状态码 %d）"
        % ("[符合]" if r3.status_code == 200 else "[不符合]",
        r3.status_code))
assert r3.status_code == 200
# ------------------------------------------------------------------
        ------------------
print()
print(LINE)
print("二、删除功能")
print(LINE)
client.post("/login", data={"username": "admin",
        "password": "admin123"})
# 项目：列表页须有删除按钮
```

```
html = client.get("/project/").get_data(as_text=True)
has_del = "/delete" in html and "btn--danger" in html
print("  %s 项目列表含删除按钮" % ("[符合]" if has_del else "[不符合]"))
assert has_del, "项目列表缺少删除按钮"
print("  %s 删除前有二次确认" % ("[符合]" if "confirm(" in html else
    "[不符合]"))
assert "confirm(" in html
# 图斑列表
with app.app_context():
    from app.models import MinePatch, Project
    import seed_demo
    patch = seed_demo.demo_patch()
    patch_id = patch.id
html = client.get("/project/%d/patches" %
    patch_id).get_data(as_text=True)
print("  %s 图斑列表含删除按钮" % ("[符合]" if "/project/patch/" in
    html and "btn--danger" in html else "[不符合]"))
assert "btn--danger" in html
# 微地质单元
html = client.get("/unit/%d" % patch_id).get_data(as_text=Tr
    ue)
print("  %s 微单元列表含删除按钮" % ("[符合]" if "/unit/delete/" in
    html else "[不符合]"))
assert "/unit/delete/" in html
print("  %s 微单元编辑为独立页面（表单不再嵌在表格行内）"
      % ("[符合]" if "/unit/edit/" in html else "[不符合]"))
# 标准
html = client.get("/standard/").get_data(as_text=True)
print("  %s 标准列表含删除按钮" % ("[符合]" if "btn--danger" in html
    else "[不符合]"))
assert "btn--danger" in html
# 实际执行一次删除：新建项目 → 删除 → 确认消失
client.post("/project/add", data={"name": "待删除测试项目",
    "code": "DEL-1"})
with app.app_context():
    from app.models import db, Project
    tmp = Project.query.filter_by(name="待删除测试项目").first()
    tmp_id = tmp.id
html = client.get("/project/").get_data(as_text=True)
print("  %s 新建项目成功" % ("[符合]" if "待删除测试项目" in html else
    "[不符合]"))
r = client.post("/project/%d/delete" % tmp_id,
    follow_redirects=True)
html = r.get_data(as_text=True)
gone = "待删除测试项目" not in html
print("  %s 删除项目后列表中不再出现" % ("[符合]" if gone else "[不符合]"))
assert gone
print("  %s 有删除成功提示" % ("[符合]" if "已删除" in html else
    "[不符合]"))
print()
```

```
print(LINE)
print("界面修复自检全部通过 √")
print(LINE)
#  ==== 测试/test_new_features.py ====
#  -*- coding: utf-8 -*-
"""补齐模块自检：操作日志与用户维护、遥感影像、指标与多专家矩阵、
批量导入、图表与 PDF 导出。
对应计划书功能清单与系统实现的差异核对结论。
"""
import io
import json
import os
import re
import sys
sys.stdout.reconfigure(encoding="utf-8")
import fitz
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, User, Project, MinePatch,
    MicroUnit, IndicatorData,
                        Indicator, ExpertMatrix,
    OperationLog)
from app.utils.schema import ensure_schema
from app.utils import browser, charts
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
    UPLOAD_FOLDER = os.path.join(BASE, "app", "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
app = create_app(TestConfig)
with app.app_context():
    db.create_all()
    ensure_schema(db)
    ExpertMatrix.query.delete()
    db.session.commit()
    #  固定取内置演示案例的图斑，避免受库中用户自建项目 / 验证算例的影响
```

```
    import seed_demo
    patch_id = seed_demo.demo_patch().id
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
def ok(flag, label, extra=""):
    print("  %s %-44s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    assert flag, label
# ------------------------------------------------------------
    ----------------
print(LINE)
print("操作日志与用户维护")
print(LINE)
before = None
with app.app_context():
    before = OperationLog.query.count()
c.post("/project/add", data={"name": "日志核对项目", "code":
    "LOG-T"})
with app.app_context():
    after = OperationLog.query.count()
    last = OperationLog.query.order_by(OperationLog.id.desc(
    )).first()
ok(after == before + 1, "写操作被写入操作日志", "%d -> %d" % (before,
    after))
ok(last.action == "新建项目", "日志记录了可读动作名", last.action)
ok("project" in (last.target_type or ""), "日志记录了对象类型",
    last.target_type)
r = c.get("/logs")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "操作日志" in body, "操作日志查看页可访问")
ok("新建项目" in body, "日志页显示操作记录")
# 用户编辑 / 删除
with app.app_context():
    u = User(username="tmp_user", role="viewer",
             password_hash=User.query.first().password_hash)
    db.session.add(u); db.session.commit()
    uid = u.id
r = c.get("/users/%d/edit" % uid)
ok(r.status_code == 200, "用户编辑页可访问")
c.post("/users/%d/edit" % uid, data={"role": "evaluator",
    "active": "1"})
with app.app_context():
    ok(db.session.get(User, uid).role == "evaluator",
    "角色已修改并保存")
# 最后一个管理员保护
with app.app_context():
    admin_id = User.query.filter_by(role="admin").first().id
c.post("/users/%d/edit" % admin_id, data={"role": "viewer",
    "active": "1"})
```

```
with app.app_context():
    ok(db.session.get(User, admin_id).role == "admin",
    "拒绝把最后一个管理员降级")
c.post("/users/%d/delete" % uid, follow_redirects=True)
with app.app_context():
    ok(db.session.get(User, uid) is None, "用户删除生效")
# 清理日志核对用的项目
with app.app_context():
    for p in Project.query.filter_by(name="日志核对项目").all():
        db.session.delete(p)
    db.session.commit()
# ------------------------------------------------------------
    ----------------
print()
print(LINE)
print("遥感影像挂接")
print(LINE)
doc = fitz.open()
pg = doc.new_page(width=320, height=200)
pg.insert_text((20, 60), "RS image", fontname="china-s",
    fontsize=16)
png = doc[0].get_pixmap(dpi=100).tobytes("png")
doc.close()
r = c.post("/project/patch/%d/image" % patch_id,
            data={"image": (io.BytesIO(png), "影像.png"),
                  "image_note": "2025年6月无人机正射影像"},
            content_type="multipart/form-data",
    follow_redirects=True)
body = r.get_data(as_text=True)
ok("已挂接" in body and "<img" in body, "影像上传并显示")
with app.app_context():
    rel = db.session.get(MinePatch, patch_id).image_path
ok(bool(rel), "数据库记录影像路径", rel)
abs_path = os.path.join(TestConfig.UPLOAD_FOLDER,
    rel.replace("uploads/", "", 1))
ok(os.path.isfile(abs_path), "影像文件落盘")
ok(c.get("/static/" + rel).status_code == 200, "影像可经 URL
    访问")
r = c.post("/project/patch/%d/image" % patch_id,
            data={"image": (io.BytesIO(b"x"), "bad.txt")},
            content_type="multipart/form-data",
    follow_redirects=True)
ok("不支持" in r.get_data(as_text=True), "拒绝非图片格式")
c.post("/project/patch/%d/image/delete" % patch_id,
    follow_redirects=True)
with app.app_context():
    ok(db.session.get(MinePatch, patch_id).image_path is
    None, "影像移除后字段清空")
ok(not os.path.isfile(abs_path), "影像文件已删除")
# ------------------------------------------------------------
```

```
    ------------------
print()
print(LINE)
print("指标定义维护与多专家矩阵")
print(LINE)
r = c.get("/ahp/indicators")
body = r.get_data(as_text=True)
with app.app_context():
    n_ind = Indicator.query.count()
ok(r.status_code == 200 and n_ind > 0, "指标定义已入库并可查看", "%d
    项" % n_ind)
ok(all(k in body for k in ("一级指标", "二级指标", "三级指标")),
   "指标树按层级展示")
ok("岩质边坡" in body and "平台" in body, "显示各指标启用的微地质单元")
c.post("/ahp/indicators/coverage/edit",
       data={"name": "植被覆盖度", "unit": "%", "ideal_type":
    "benefit",
             "ideal_value": "75", "enabled_units": ["岩质边坡"],
             "description": "自检修改"})
with app.app_context():
    ok(Indicator.query.filter_by(code="coverage").first().de
    scription == "自检修改",
       "指标定义可编辑")
c.post("/ahp/indicators/reset")
with app.app_context():
    ok("自检修改" not in (Indicator.query.filter_by(code="covera
    ge").first().description or ""),
       "可按论文取值重置")
base = {"m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24"}
for name in ("张工", "李工", "王工"):
    d = dict(base); d.update({"action": "add", "unit_type":
    "碎石边坡",
                              "level": "1", "expert_name":
    name})
    c.post("/ahp/experts", data=d)
with app.app_context():
    ok(ExpertMatrix.query.filter_by(unit_type="碎石边坡",
    level=1).count() == 3,
       "录入 3 位专家的判断矩阵")
r = c.get("/ahp/experts/aggregate?unit_type=碎石边坡&level=1")
body = r.get_data(as_text=True)
ok("汇总后权重" in body and "各专家矩阵的一致性" in body,
   "多专家矩阵几何平均汇总可用")
ws = re.search(r"<th scope=\"row\">权重</th>\s*(.*?)</tr>",
    body, re.S)
vals = re.findall(r">([0-9.]+)</td>", ws.group(1)) if ws
    else []
ok(len(vals) == 3 and abs(sum(map(float, vals)) - 1.0) <
    0.01,
   "汇总权重归一", "、".join(vals))
```

```
ok(abs(float(vals[0]) - 0.44) < 0.02, "汇总权重接近论文表 3 取值",
   vals[0] if vals else "")
# ----------------------------------------------------------------
   ----------------
print()
print(LINE)
print("批量导入")
print(LINE)
r = c.get("/indicator/import/template")
tpl = r.data.decode("utf-8-sig")
ok(r.status_code == 200 and "单元类型" in tpl, "导入模板可下载")
ok("土石比" in tpl and "植被覆盖度" in tpl, "模板包含指标列")
csv_text = (
   "单元类型,土石比,土层厚度,pH,有机质,Hg,As,Ni,Cu,Cd,Pb,植被覆盖度,Shannon,坡度
   ,坡向\n"
   "碎石边坡,0.34,10,9.24,4.89,0.05,5.17,12.60,11.70,0.04,6.50,
   87.92,0.60,28.75,阳坡\n"
   "平台,0.56,30,8.72,10.92,0.02,3.13,12.33,11.14,0.09,9.98,5
   8.42,0.72,,,\n"
   "不存在单元,1,1,7,1,,,,,,,,,\n"
)
r = c.post("/indicator/import",
           data={"patch_id": str(patch_id),
                 "datafile": (io.BytesIO(csv_text.encode("ut
   f-8")), "d.csv")},
           content_type="multipart/form-data",
   follow_redirects=True)
body = r.get_data(as_text=True)
ok("导入结果" in body, "导入结果页展示")
ok("不存在单元" in body, "错误行被报告")
with app.app_context():
   u = MicroUnit.query.filter_by(patch_id=patch_id,
   unit_type="碎石边坡").first()
   ind = IndicatorData.query.filter_by(unit_id=u.id).first(
   )
   ok(abs(ind.soil_ratio - 0.34) < 1e-6 and
   abs(ind.coverage - 87.92) < 1e-6,
       "导入数据已写入数据库")
r = c.post("/indicator/import",
           data={"patch_id": str(patch_id),
                 "datafile": (io.BytesIO(b"x"), "d.pdf")},
           content_type="multipart/form-data",
   follow_redirects=True)
ok("仅支持" in r.get_data(as_text=True), "拒绝不支持的导入格式")
# ----------------------------------------------------------------
   ----------------
print()
print(LINE)
print("图表与 PDF 导出")
print(LINE)
```

```
svg = charts.hbars([("岩质边坡", 0.54, "0.5401"), ("平台", 0.84,
    "0.8376")])
ok("<svg" in svg and "0.8376" in svg, "条形图生成且带数值标签")
svg2 = charts.donut([("植被指标", 0.6, "0.6000"), ("土壤指标", 0.4,
    "0.4000")])
ok("<svg" in svg2 and "占比" in svg2, "环形图生成且带图例")
#  报告是评价结论的呈现，需先真实执行一次评价才能打开
with app.app_context():
    from app.views.evaluate import load_patch_input,
    save_evaluation
    save_evaluation(patch_id, load_patch_input(patch_id))
r = c.get("/report/%d" % patch_id)
body = r.get_data(as_text=True)
ok(body.count("<svg") >= 2, "得分明细页含图表", "%d 个 SVG" %
    body.count("<svg"))
exe = browser.browser_path()
if exe is None:
    print("  [跳过] 未找到浏览器，PDF 导出无法验证")
else:
    r = c.get("/report/%d/pdf" % patch_id)
    data = r.data
    ok(data[:4] == b"%PDF", "PDF 报告导出成功", "%d 字节" %
    len(data))
    d = fitz.open(stream=data, filetype="pdf")
    txt = "".join(d[i].get_text() for i in
    range(d.page_count))
    ok("自然恢复适宜性评价报告" in txt, "PDF 含报告标题")
    ok("评价结论" in txt and "得分明细" in txt, "PDF 含结论与明细章节")
    ok(d.page_count >= 2, "PDF 页数合理", "%d 页" % d.page_count)
    d.close()
print()
print(LINE)
print("补齐模块自检全部通过 √")
print(LINE)
#  ==== 测试/test_workflow.py ====
#  -*- coding: utf-8 -*-
"""流程门禁自检：前置步骤未完成时，后续模块必须拦得住。
论文 4.1 节确定的评价流程存在先后关系，缺任何一环都不应产出评价结论。
本测试逐个构造「缺前置」的场景，验证系统不会放行。
"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
```

```
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult)
from app.services import workflow as wf
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    if not flag:
        _fails.append(label)
def new_patch(tag):
    with app.app_context():
        p = Project(name="门禁-" + tag, code="WF-" + tag)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="门禁图斑-" +
    tag)
        db.session.add(patch); db.session.flush()
        pid = patch.id
        db.session.commit()
    return pid
def add_units(pid):
    with app.app_context():
        db.session.add(MicroUnit(patch_id=pid,
    unit_type="碎石边坡",
                                 area=1.0, ratio=1.0))
        db.session.commit()
def add_constraint(pid, passed=True):
    with app.app_context():
        db.session.add(ConstraintCheck(
            patch_id=pid, policy_pass=True,
    location_pass=passed,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=passed, reason="全部通过" if passed else
    "未通过：区位约束"))
        db.session.commit()
def add_data(pid):
```

```
    with app.app_context():
        u = MicroUnit.query.filter_by(patch_id=pid).first()
        db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.34, thickness=10,
                                     ph=9.24,
    organic_matter=4.89, coverage=87.92,
                                     shannon=0.60,
    slope=28.75, aspect="阳坡"))
        db.session.commit()
def cleanup(pid, tag):
    with app.app_context():
        from app.models import ScoreDetail
        # 先删得分明细，再删评价记录，避免留下孤儿明细
        rids = [r.id for r in EvaluationResult.query.filter_
    by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.i
    n_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delet
    e()
        ConstraintCheck.query.filter_by(patch_id=pid).delete
    ()
        uids = [u.id for u in MicroUnit.query.filter_by(patc
    h_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id
    .in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
        m = db.session.get(MinePatch, pid)
        if m:
            db.session.delete(m)
        pr = Project.query.filter_by(name="门禁-" +
    tag).first()
        if pr:
            db.session.delete(pr)
        db.session.commit()
def results_of(pid):
    with app.app_context():
        return EvaluationResult.query.filter_by(patch_id=pid
    ).count()
def post_eval(pid):
    return c.post("/evaluate/%d" % pid, data={},
    follow_redirects=False)
def get_report(pid):
    return c.get("/report/%d" % pid, follow_redirects=False)
# ----------------------------------------------------------------
    ----------------
print(LINE)
```

```
print("一、前置条件缺失时，评价必须被拦住")
print(LINE)
print()
print("场景　1：连微地质单元都没划分")
pid = new_patch("1")
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未划分单元 → 不产生评价结果",
   "HTTP %d，结果数 %d" % (r.status_code, n))
with app.app_context():
    pr = wf.patch_progress(pid)
steps = [b["step"] for b in pr["blocking"]]
ok(not pr["ready"] and "units" in steps,
   "拦原因包含「划分微地质单元」", "、".join(steps))
ok(steps == ["constraint", "units"],
   "拦截顺序符合论文 4.1 节：限制性判定在前，划分单元在后",
   "、".join(steps))
cleanup(pid, "1")
print()
print("场景　2：有单元，但未做限制性条件判定")
pid = new_patch("2"); add_units(pid); add_data(pid)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未做限制性判定 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "constraint" for b in pr["blocking"]),
   "拦原因指向「执行限制性条件判定」")
cleanup(pid, "2")
print()
print("场景　3：限制性条件判定【未通过】")
pid = new_patch("3"); add_units(pid); add_data(pid);
    add_constraint(pid, passed=False)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "判定未通过 → 不进入定量评价",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(not pr["constraint"]["passed"], "流程状态显示判定未通过")
blocked = [b for b in pr["blocking"] if b["step"] ==
    "constraint"]
ok(bool(blocked) and "人工辅助" in blocked[0]["reason"],
   "拦原因说明了按论文应判「人工辅助或生态重建」")
cleanup(pid, "3")
print()
print("场景　4：判定通过，但指标数据全空")
pid = new_patch("4"); add_units(pid); add_constraint(pid,
    passed=True)
```

```
r = post_eval(pid)                                                    8619
n = results_of(pid)                                                   8620
ok(r.status_code == 302 and n == 0, "数据未录入 → 不产生评价结果",      8621
   "结果数 %d" % n)                                                   8622
with app.app_context():                                               8623
    pr = wf.patch_progress(pid)                                       8624
ok(any(b["step"] == "data" for b in pr["blocking"]),                  8625
   "拦原因指向「录入指标数据」")                                      8626
ok(pr["data"]["missing"] == ["碎石边坡"],                             8627
   "指出具体缺哪个单元的数据", "、".join(pr["data"]["missing"]))      8628
cleanup(pid, "4")                                                     8629
print()                                                               8630
print("场景 5：单元面积占比合计不为 100%")                           8631
pid = new_patch("5"); add_constraint(pid, passed=True)                8632
with app.app_context():                                               8633
    u = MicroUnit(patch_id=pid, unit_type="碎石边坡", area=1.0,       8634
    ratio=0.5)                                                        8635
    db.session.add(u); db.session.commit()                            8636
    db.session.add(IndicatorData(unit_id=u.id,                        8637
    soil_ratio=0.3, thickness=10,                                     8638
                                          ph=8.0,                     8639
    organic_matter=5.0, coverage=80.0,                                8640
                                          shannon=0.6, slope=20.0,    8641
    aspect="阳坡"))                                                   8642
    db.session.commit()                                               8643
r = post_eval(pid)                                                    8644
ok(results_of(pid) == 0, "占比不足 100% → 不产生评价结果")            8645
cleanup(pid, "5")                                                     8646
print()                                                               8647
print(LINE)                                                           8648
print("二、前置条件齐备时才放行")                                     8649
print(LINE)                                                           8650
pid = new_patch("OK"); add_units(pid); add_constraint(pid,            8651
    passed=True); add_data(pid)                                       8652
with app.app_context():                                               8653
    pr = wf.patch_progress(pid)                                       8654
ok(pr["ready"], "单元 + 判定通过 + 数据齐备 → 判定为可执行")          8655
r = post_eval(pid)                                                    8656
n = results_of(pid)                                                   8657
ok(n == 1, "此时执行评价成功产生结果", "结果数 %d" % n)               8658
print("  重定向到：", r.headers.get("Location"))                      8659
print()                                                               8660
print(LINE)                                                           8661
print("三、报告不得先于评价产生")                                     8662
print(LINE)                                                           8663
pid2 = new_patch("R")    # 什么都不做                                 8664
r = get_report(pid2)                                                  8665
ok(r.status_code == 302 and "/report/" ==                             8666
    (r.headers.get("Location") or ""),                                8667
   "前置缺失时 /report/<id> 被退回列表", "HTTP %d" % r.status_code)   8668
```

```
for suffix in ("/pdf", "/word"):
    rr = c.get("/report/%d%s" % (pid2, suffix),
    follow_redirects=False)
    ok(rr.status_code == 302, "导出 %s 同样被拦住" % suffix, "HTTP
    %d" % rr.status_code)
cleanup(pid2, "R")
cleanup(pid, "OK")
print()
print(LINE)
print("四、前置齐备但从未执行评价时，报告仍不得产生")
print(LINE)
#  报告是评价结论的呈现：即使前置步骤都做完了，只要没真正执行过定量评价，
#  也不应凭空推算出结论。
pid3 = new_patch("N")
add_units(pid3); add_constraint(pid3, passed=True);
    add_data(pid3)
with app.app_context():
    pr = wf.report_progress(pid3)
ok(not pr["ready"], "无评价记录 → 报告流程判定为未就绪")
ok(bool(pr["blocking"]) and pr["blocking"][0]["step"] ==
    "evaluate",
    "拦原因指向「执行适宜性评价」")
r = get_report(pid3)
ok(r.status_code == 302 and results_of(pid3) == 0,
    "直接访问 /report/<id> 被退回，且不产生任何评价结果",
    "HTTP %d" % r.status_code)
#  真正执行评价之后，报告才放行，且取值与评价记录一致
r = post_eval(pid3)
ok(results_of(pid3) == 1, "先执行评价 → 产生评价记录")
with app.app_context():
    pr = wf.report_progress(pid3)
    rec = pr["result"]
ok(pr["ready"] and rec is not None, "有评价记录后，报告流程就绪")
rr = get_report(pid3)
body = rr.get_data(as_text=True)
ok(rr.status_code == 200, "此时可打开得分明细页", "HTTP %d" %
    rr.status_code)
ok(("%.4f" % rec.final_score) in body,
    "报告数值取自评价记录，而非重新计算",
    "F=%.4f" % rec.final_score)
cleanup(pid3, "N")
print()
print(LINE)
print("五、内置演示数据的清理不得波及用户自建数据")
print(LINE)
#  回归保护：seed_demo.clean_case() 曾执行 Project.query.delete()，
#  导致用户新建的项目在每次启动时被连带删除。
import seed_demo as sd
with app.app_context():
    keep_p = Project(name="用户自建项目-守护", code="KEEP-001")
```

```
    db.session.add(keep_p); db.session.flush()
    keep_patch = MinePatch(project_id=keep_p.id,
    name="用户自建图斑 - 守护")
    db.session.add(keep_patch); db.session.flush()
    keep_pid = keep_p.id
    keep_patch_id = keep_patch.id
    db.session.commit()
with app.app_context():
    sd.clean_case(reset=False)
with app.app_context():
    still_p = db.session.get(Project, keep_pid)
    still_patch = db.session.get(MinePatch, keep_patch_id)
ok(still_p is not None, "清理演示数据后，用户自建项目仍存在")
ok(still_patch is not None, "清理演示数据后，用户自建图斑仍存在")
# 演示数据清掉后应能重新写入，保证演示环境可恢复
sd.seed()
with app.app_context():
    demo_back = Project.query.filter_by(code=sd.DEMO_CODE).f
    irst()
ok(demo_back is not None, "演示案例可重新写入（幂等恢复）")
with app.app_context():
    m = db.session.get(MinePatch, keep_patch_id)
    if m:
        db.session.delete(m)
    p = db.session.get(Project, keep_pid)
    if p:
        db.session.delete(p)
    db.session.commit()
print()
print(LINE)
print("六、同一图斑不得重复划分同类型微地质单元")
print(LINE)
# 论文 4.1 节把图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。
# 允许重复会让面积占比、按类型组织的指标数据与批量导入对不上号。
pid4 = new_patch("D")
c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
    "area": "1.0"},
       follow_redirects=True)
with app.app_context():
    n1 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n1 == 1, "首次添加「平台」成功", "单元数 %d" % n1)
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
    "area": "2.0"},
              follow_redirects=True)
with app.app_context():
    n2 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n2 == 1, "重复添加同类型被拒绝", "单元数 %d" % n2)
ok("同一类型只能划分一个" in r.get_data(as_text=True), "给出重复提示")
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "火山口",
    "area": "1.0"},
```

```
            follow_redirects=True)                                    8769
with app.app_context():                                               8770
    n3 = MicroUnit.query.filter_by(patch_id=pid4).count()             8771
ok(n3 == 1, "非法单元类型被拒绝", "单元数 %d" % n3)                    8772
c.post("/unit/%d/add" % pid4, data={"unit_type": "碎石边坡",           8773
    "area": "3.0"},                                                   8774
       follow_redirects=True)                                         8775
with app.app_context():                                               8776
    uid = MicroUnit.query.filter_by(patch_id=pid4,                    8777
                                    unit_type="碎石边坡").first(      8778
    ).id                                                              8779
r = c.post("/unit/edit/%d" % uid,                                     8780
            data={"unit_type": "平台", "area": "3.0"},                8781
    follow_redirects=True)                                            8782
with app.app_context():                                               8783
    types = sorted(u.unit_type for u in                               8784
                   MicroUnit.query.filter_by(patch_id=pid4).          8785
    all())                                                            8786
ok(types == ["平台", "碎石边坡"], "改类型时同样拦截重复", "、".join(types))  8787
cleanup(pid4, "D")                                                    8788
print()                                                               8789
print(LINE)                                                           8790
if _fails:                                                            8791
    print("存在未通过项：")                                           8792
    for f in _fails:                                                  8793
        print("  -", f)                                              8794
    sys.exit(1)                                                       8795
print("流程门禁自检全部通过 √")                                      8796
print(LINE)                                                           8797
# ==== 测试/test_weights.py ====                                      8798
# -*- coding: utf-8 -*-                                               8799
"""权重生效链路自检：AHP 算出的权重必须真正参与评价计算。             8800
此前 weight_schemes / weight_items 两张表从未被写入，评价引擎也只用   8801
代码里的论文表 3 默认权重，相当于权重模块算完即弃。本测试守住这条链路：  8802
    默认 → 论文表 3                                                   8803
    保存 AHP 权重 → 评价改用新权重，结果随之变化                      8804
    恢复默认 → 结果回到论文取值                                       8805
"""                                                                   8806
import os                                                             8807
import sys                                                            8808
sys.stdout.reconfigure(encoding="utf-8")                              8809
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，           8810
# 使 `from app import ...` 与 app/app.db 的定位不受影响。             8811
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil          8812
    e__)))                                                            8813
if ROOT not in sys.path:                                              8814
    sys.path.insert(0, ROOT)                                          8815
from app import create_app                                            8816
from app.models import (db, Project, MinePatch, MicroUnit,            8817
    IndicatorData,                                                    8818
```

```
                                ConstraintCheck, EvaluationResult,
    WeightScheme,
                                WeightItem)
from app.services import workflow as wf
from app.services import weight_service as ws
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    if not flag:
        _fails.append(label)
TAG = "W"
UNIT_TYPE = "碎石边坡"
def build_case():
    """建一个只含单一微地质单元的图斑，便于观察权重变化对得分的影响。"""
    with app.app_context():
        p = Project(name="权重测试-%s" % TAG, code="WEIGHT-%s"
    % TAG)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="权重测试图斑-%s"
    % TAG)
        db.session.add(patch); db.session.flush()
        db.session.add(MicroUnit(patch_id=patch.id,
    unit_type=UNIT_TYPE,
                                 area=1.0, ratio=1.0))
        db.session.flush()
        u = MicroUnit.query.filter_by(patch_id=patch.id).fir
    st()
        db.session.add(IndicatorData(
            unit_id=u.id, soil_ratio=0.34, thickness=10,
    ph=9.24,
            organic_matter=4.89, coverage=87.92,
    shannon=0.60,
            slope=28.75, aspect="阳坡"))
        db.session.add(ConstraintCheck(
            patch_id=patch.id, policy_pass=True,
    location_pass=True,
```

```
                pollution_pass=True, geo_pass=True,
    farmland_pass=True,
                passed=True, reason="全部通过"))
            db.session.commit()
            return patch.id
def cleanup(pid):
    with app.app_context():
        from app.models import ScoreDetail
        rids = [r.id for r in EvaluationResult.query.filter_
    by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.i
    n_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delet
    e()
        ConstraintCheck.query.filter_by(patch_id=pid).delete
    ()
        uids = [u.id for u in MicroUnit.query.filter_by(patc
    h_id=pid)]
        if uids:
            IndicatorData.query.filter(IndicatorData.unit_id
    .in_(uids)).delete(
                synchronize_session=False)
        MicroUnit.query.filter_by(patch_id=pid).delete()
        m = db.session.get(MinePatch, pid)
        if m:
            db.session.delete(m)
        p = Project.query.filter_by(code="WEIGHT-%s" %
    TAG).first()
        if p:
            db.session.delete(p)
        db.session.commit()
def run_eval(pid):
    c.post("/evaluate/%d" % pid, data={},
    follow_redirects=False)
    with app.app_context():
        return EvaluationResult.query.filter_by(patch_id=pid
    ) \
            .order_by(EvaluationResult.id.desc()).first().fi
    nal_score
print(LINE)
print("一、默认应使用论文表 3 的权重")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
    w, label, scheme = ws.effective_weights(UNIT_TYPE)
ok(scheme is None and "论文表" in label, "未配置时权重来源为论文表 3",
    label)
ok(abs(w["l1"]["土壤指标"] - 0.44) < 1e-9, "一级权重取自论文表 3",
```

```
    "土壤指标 %.2f" % w["l1"]["土壤指标"])                                    8919
pid = build_case()                                                          8920
base_score = run_eval(pid)                                                  8921
print("   默认权重下的评价值 F = %.4f" % base_score)                          8922
print()                                                                     8923
print(LINE)                                                                 8924
print("二、保存 AHP 算出的权重后，评价应改用新权重")                          8925
print(LINE)                                                                 8926
with app.app_context():                                                     8927
    # 模拟 AHP 计算结果：把土壤指标权重提高、植被与地形降低（仍和为 1）          8928
    ws.save_weights(UNIT_TYPE, [("soil", 0.70), ("veg",                     8929
    0.20),                                                                  8930
                                ("terrain", 0.10)])                         8931
    n_items = WeightItem.query.count()                                      8932
ok(n_items >= 3, "weight_items 表已写入权重条目", "%d 条" % n_items)         8933
with app.app_context():                                                     8934
    w2, label2, scheme2 = ws.effective_weights(UNIT_TYPE)                   8935
ok(scheme2 is not None and "自定义" in label2, "权重来源变为自定义方案",      8936
    label2)                                                                 8937
ok(abs(w2["l1"]["土壤指标"] - 0.70) < 1e-9, "新权重已生效",                  8938
    "土壤指标 %.2f" % w2["l1"]["土壤指标"])                                  8939
ok(abs(w2["l1"]["植被指标"] - 0.20) < 1e-9, "同批保存的其他项也已生效")       8940
ok(abs(w2["l2"]["土壤指标"]["土石比"] - 0.25) < 1e-9,                        8941
    "未覆盖的二级权重仍保留论文取值",                                        8942
    "土石比 %.2f" % w2["l2"]["土壤指标"]["土石比"])                          8943
new_score = run_eval(pid)                                                   8944
print("   新权重下的评价值 F = %.4f" % new_score)                             8945
ok(abs(new_score - base_score) > 1e-4,                                      8946
    "权重变化确实改变了评价结果",                                            8947
    "%.4f → %.4f" % (base_score, new_score))                                8948
print()                                                                     8949
print(LINE)                                                                 8950
print("三、结果应记录所用权重来源")                                          8951
print(LINE)                                                                 8952
import json                                                                 8953
with app.app_context():                                                     8954
    rec = EvaluationResult.query.filter_by(patch_id=pid) \                  8955
        .order_by(EvaluationResult.id.desc()).first()                       8956
    stored = json.loads(rec.unit_scores)                                    8957
ok(any(u.get("weight_source") for u in stored),                             8958
    "评价记录中标注了权重来源",                                              8959
    stored[0].get("weight_source") or "")                                   8960
print()                                                                     8961
print(LINE)                                                                 8962
print("四、恢复默认后应回到论文取值")                                        8963
print(LINE)                                                                 8964
with app.app_context():                                                     8965
    ws.reset_weights(UNIT_TYPE)                                             8966
    w3, label3, scheme3 = ws.effective_weights(UNIT_TYPE)                   8967
ok(scheme3 is None and "论文表" in label3, "已恢复为论文表 3 权重",          8968
```

```
    label3)                                                                 8969
back_score = run_eval(pid)                                                  8970
print("   恢复后评价值 F = %.4f（默认时 %.4f）" % (back_score,               8971
    base_score))                                                            8972
ok(abs(back_score - base_score) < 1e-9, "评价结果与默认时一致")             8973
print()                                                                     8974
print(LINE)                                                                 8975
print("五、权重首页概览")                                                   8976
print(LINE)                                                                 8977
with app.app_context():                                                     8978
    rows = ws.summary()                                                     8979
ok(len(rows) == 3, "三种微地质单元均有权重来源说明", "%d 行" % len(rows))   8980
r = c.get("/ahp/")                                                          8981
body = r.get_data(as_text=True)                                             8982
ok(r.status_code == 200 and "当前生效权重" in body, "权重首页正常展示")     8983
print()                                                                     8984
print(LINE)                                                                 8985
print("六、一致性不通过的矩阵不得保存生效")                                 8986
print(LINE)                                                                 8987
with app.app_context():                                                     8988
    ws.reset_weights(UNIT_TYPE)                                             8989
n = 3                                                                       8990
# 明显矛盾的矩阵：a12=9, a13=1/9, a23=9 —— CR 必然超限                      8991
mats = [(0, 1, "9"), (0, 2, "0.1111"), (1, 2, "9")]                         8992
data = {"csrf_token": "", "action": "save"}                                 8993
for i, j, v in mats:                                                        8994
    data["m_%d_%d" % (i, j)] = v                                            8995
r = c.post("/ahp/matrix?unit_type=%s&level=1" % UNIT_TYPE,                  8996
    data=data,                                                              8997
              follow_redirects=True)                                        8998
with app.app_context():                                                     8999
    _, label4, scheme4 = ws.effective_weights(UNIT_TYPE)                    9000
ok(scheme4 is None, "CR 超限的矩阵未被保存生效", label4)                    9001
ok("一致性" in r.get_data(as_text=True), "页面给出了未通过一致性的提示")    9002
cleanup(pid)                                                                9003
print()                                                                     9004
print(LINE)                                                                 9005
if _fails:                                                                  9006
    print("存在未通过项：")                                                 9007
    for f in _fails:                                                        9008
        print("  -", f)                                                    9009
    sys.exit(1)                                                             9010
print("权重生效链路自检全部通过 √")                                        9011
print(LINE)                                                                 9012
# ==== app/services/__init__.py ====                                        9013
# -*- coding: utf-8 -*-                                                     9014
# ==== app/utils/__init__.py ====                                           9015
# -*- coding: utf-8 -*-                                                     9016
# ==== app/views/__init__.py ====                                           9017
# -*- coding: utf-8 -*-                                                     9018
```